Casos de Uso Ortomapas v2.0

Casos de Uso - Sistema de Ortomapas

Versao: 2.0 - revisada para aderencia da interface **Data:** 2026-09-15 **Sistema:** Ortomapas - FastAPI + Vue3 + Leaflet
Backend: http://localhost:8888 | **Frontend:** http://localhost:5176

> Esta versao substitui a classificacao otimista da versao 1.0. Ela distingue requisito implementado no backend de caso de uso executavel pela interface.

Indice

ID	Nome	Modulo	Prioridade
UC-001	Criar Novo Projeto	Projetos	Alta
UC-002	Buscar Projetos por Texto e Filtrar por Status	Projetos	Alta
UC-003	Selecionar Projeto e Visualizar Ortomapas	Projetos	Alta
UC-004	Fazer Upload de GeoTIFF e Visualizar no Mapa	Ortomapas	Alta
UC-005	Calcular Indice de Vegetacao VARI	Analise	Alta
UC-006	Calcular Declividade (Slope) a partir de DSM	Analise	Alta
UC-007	Gerar Curvas de Nivel a partir de DSM	Analise	Media
UC-008	Gerar Hillshade a partir de DSM	Analise	Media
UC-009	Detectar Mudancas Entre Dois Ortomapas	Analise	Alta
UC-010	Classificar Uso do Solo com KMeans	Analise	Alta
UC-011	Extrair Rede de Drenagem (Hidrologia)	Analise	Media
UC-012	Calcular Volume Acima/Abaixo de Referencia	Analise	Media
UC-013	Criar Anotacao Poligono no Mapa	Anotacoes	Alta
UC-014	Criar Anotacao Ponto no Mapa	Anotacoes	Alta
UC-015	Medir Distancia no Mapa	Mapa	Media
UC-016	Medir Area no Mapa	Mapa	Media
UC-017	Comparar Dois Ortomapas (Swipe)	Mapa	Media
UC-018	Exportar Ortomapa em Formato Diferente	Exportacao	Alta
UC-019	Registrar Voo de Drone	Projetos	Media
UC-020	Disparar Analise via Agente de IA	Analise	Alta

UC-001: Criar Novo Projeto

Identificacao

- **ID:** UC-001
- **Modulo:** Projetos

- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Criar um novo projeto de mapeamento no sistema, definindo nome, descricao e area de estudo, para que o usuario possa organizar seus ortomapas, voos e analises sob esse projeto.

Pre-condicoes

- O sistema esta acessivel em http://localhost:5176
- O backend esta respondendo em http://localhost:8888
- A sidebar esquerda esta visivel (nao colapsada)

Pos-condicoes

- Um novo projeto existe no banco de dados com status "ativo"
- O projeto aparece na lista de projetos na sidebar esquerda
- O dialog de criacao foi fechado
- Os campos do formulario foram resetados

Elementos de UI Envolvidos

- Botao "Novo" (Button PrimeVue, icon `pi pi-plus`, severity `success`) na secao Projetos
- Dialog modal "Novo Projeto" (Dialog PrimeVue, largura 450px)
- InputText "Nome do Projeto" (placeholder "Nome")
- Textarea "Descricao" (3 linhas)
- InputText "Area de Estudo" (placeholder "Ex: Fazenda Norte")
- Botao "Cancelar" (severity `secondary`)
- Botao "Criar" (icon `pi pi-check`)
- Lista `.project-item` na sidebar

Fluxo Principal

Passo 1 - Abrir o dialog de novo projeto

- **Acao do ator:** Clica no botao "Novo" ao lado do titulo "Projetos" na sidebar esquerda
- **Contexto visual anterior:** Sidebar esquerda mostra a secao "Projetos" com titulo e botao "Novo" verde. Pode haver zero ou mais projetos listados abaixo
- **Elemento de UI utilizado:** Button PrimeVue com label "Novo", icon `pi pi-plus`, severity `success`, tamanho `small`, localizado dentro de `.section-header` em `.project-list`
- **Resposta imediata do sistema:** O dialog modal "Novo Projeto" aparece centralizado na tela com overlay escuro
- **Processamento funcional:** Nenhum - apenas seta `showNewProject = true` no estado local do componente ProjectList
- **Efeito observavel na UI:** Modal aparece com tres campos vazios (Nome, Descricao, Area de Estudo) e dois botoes no footer (Cancelar, Criar)
- **Evidencia esperada:** Screenshot do dialog modal aberto com campos vazios
- **Criterio de aceitacao:** O dialog deve estar visivel, com header "Novo Projeto", tres campos de input e dois botoes no footer
- **Seletor Playwright:** `page.locator('.project-list .section-header button').filter({ hasText: 'Novo' })`

Passo 2 - Preencher o nome do projeto

- **Acao do ator:** Digita "Fazenda Serra do Cipo - Levantamento 2026" no campo "Nome do Projeto"

- **Contexto visual anterior:** Dialog aberto com todos os campos vazios
- **Elemento de UI utilizado:** InputText PrimeVue dentro do primeiro `.form-group` do Dialog, com placeholder "Nome" e classe `w-full`
- **Resposta imediata do sistema:** O texto aparece no campo conforme o usuario digita
- **Processamento funcional:** O v-model `newProject.nome` e atualizado reativamente
- **Efeito observavel na UI:** O campo de nome mostra o texto digitado
- **Evidencia esperada:** Screenshot mostrando o campo nome preenchido
- **Criterio de aceitacao:** O valor do campo deve conter exatamente "Fazenda Serra do Cipo - Levantamento 2026"
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').first().locator('input')`

Passo 3 - Preencher a descricao

- **Acao do ator:** Digita "Levantamento aerofotogrametrico da fazenda para monitoramento de vegetacao e erosao" no campo "Descricao"
- **Contexto visual anterior:** Campo nome ja preenchido, campo descricao vazio
- **Elemento de UI utilizado:** Textarea PrimeVue com 3 linhas, classe `w-full`, dentro do segundo `.form-group`
- **Resposta imediata do sistema:** O texto aparece no textarea conforme digitacao
- **Processamento funcional:** O v-model `newProject.descricao` e atualizado
- **Efeito observavel na UI:** Textarea mostra o texto completo
- **Evidencia esperada:** Screenshot mostrando nome e descricao preenchidos
- **Criterio de aceitacao:** O textarea deve conter o texto informado
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').nth(1).locator('textarea')`

Passo 4 - Preencher a area de estudo

- **Acao do ator:** Digita "Serra do Cipo, MG" no campo "Area de Estudo"
- **Contexto visual anterior:** Nome e descricao ja preenchidos
- **Elemento de UI utilizado:** InputText PrimeVue com placeholder "Ex: Fazenda Norte", classe `w-full`, dentro do terceiro `.form-group`
- **Resposta imediata do sistema:** Texto aparece no campo
- **Processamento funcional:** O v-model `newProject.area_estudo` e atualizado
- **Efeito observavel na UI:** Campo exibe "Serra do Cipo, MG"
- **Evidencia esperada:** Screenshot do formulario completo preenchido
- **Criterio de aceitacao:** Todos os tres campos devem estar preenchidos corretamente
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').nth(2).locator('input')`

Passo 5 - Clicar no botao Criar

- **Acao do ator:** Clica no botao "Criar" no footer do dialog
- **Contexto visual anterior:** Formulario completamente preenchido, botao "Criar" habilitado
- **Elemento de UI utilizado:** Button PrimeVue com label "Criar", icon `pi pi-check`, no `#footer` do Dialog
- **Resposta imediata do sistema:** O dialog fecha imediatamente
- **Processamento funcional:** (1) Valida que `newProject.nome` nao esta vazio. (2) Chama `projectStore.createProject(newProject)` que faz POST para `/api/projetos` com payload `{nome, descricao, area_estudo}`. (3) O backend cria o registro no banco SQLite. (4) O store adiciona o projeto retornado ao array `projects`. (5) Reseta `newProject` para `{nome: '', descricao: '', area_estudo: ''}`. (6) Seta `showNewProject = false`
- **Efeito observavel na UI:** (1) O dialog desaparece. (2) Um novo `.project-item` aparece na lista de projetos com nome "Fazenda Serra do Cipo - Levantamento 2026", area "Serra do Cipo, MG" e badge "ativo"
- **Evidencia esperada:** Screenshot da sidebar mostrando o novo projeto na lista
- **Criterio de aceitacao:** O projeto deve aparecer na lista com nome, area e status corretos. O dialog deve estar fechado. A API deve retornar status 200/201

- **Seletor Playwright:** `page.locator('.p-dialog-footer button').filter({ hasText: 'Criar' })`

Passo 6 - Verificar o projeto na lista

- **Acao do ator:** Observa a lista de projetos na sidebar
- **Contexto visual anterior:** Dialog fechado, lista de projetos atualizada
- **Elemento de UI utilizado:** `.project-item` com `.project-name`, `.project-area`, `.project-badge`
- **Resposta imediata do sistema:** N/A - verificacao visual
- **Processamento funcional:** Nenhum - o projeto ja foi adicionado ao store
- **Efeito observavel na UI:** O novo projeto aparece na lista com: nome em `.project-name`, area em `.project-area`, badge "ativo" com severity `success` (cor verde)
- **Evidencia esperada:** Screenshot final da lista com o projeto visivel
- **Criterio de aceitacao:** Deve existir um `.project-item` cujo `.project-name` contenha "Fazenda Serra do Cipo - Levantamento 2026" e cujo `.project-area` contenha "Serra do Cipo, MG"
- **Seletor Playwright:** `page.locator('.project-item .project-name').filter({ hasText: 'Fazenda Serra do Cipo' })`

Fluxos Alternativos

FA-1: Cancelar a criacao do projeto

- **Passo divergente:** Passo 5
- **Acao do ator:** Clica no botao "Cancelar" em vez de "Criar"
- **Contexto visual anterior:** Formulario preenchido parcial ou totalmente
- **Elemento de UI utilizado:** Button PrimeVue com label "Cancelar", severity `secondary`
- **Resposta imediata do sistema:** O dialog fecha
- **Processamento funcional:** Seta `showNewProject = false`. Os dados do formulario permanecem em memoria (nao sao resetados ate a proxima abertura com criacao)
- **Efeito observavel na UI:** Dialog desaparece, nenhum novo projeto e adicionado a lista
- **Criterio de aceitacao:** Nenhum projeto novo deve aparecer na lista. O dialog deve estar fechado
- **Seletor Playwright:** `page.locator('.p-dialog-footer button').filter({ hasText: 'Cancelar' })`

FA-2: Fechar o dialog clicando no X

- **Passo divergente:** Passo 5
- **Acao do ator:** Clica no botao X (fechar) no header do Dialog PrimeVue
- **Processamento funcional:** Dialog PrimeVue seta `v-model:visible` para false
- **Efeito observavel na UI:** Dialog desaparece sem criar projeto

Fluxos de Excecao

FE-1: Nome vazio

- **Passo divergente:** Passo 5
- **Condicao:** O campo "Nome do Projeto" esta vazio
- **Acao do ator:** Clica em "Criar" com nome vazio
- **Processamento funcional:** A funcao `createProject()` verifica `if (!newProject.value.nome) return` e retorna sem fazer nada
- **Efeito observavel na UI:** O dialog permanece aberto, nenhuma mensagem de erro e exibida (retorno silencioso)
- **Criterio de aceitacao:** Nenhuma requisicao HTTP deve ser feita. O dialog deve permanecer aberto

FE-2: Erro de rede/backend

- **Passo divergente:** Passo 5
- **Condicao:** O backend esta indisponivel ou retorna erro 500
- **Processamento funcional:** `projectStore.createProject()` captura o erro, seta `this.error = e.message` e faz `throw e`
- **Efeito observavel na UI:** O dialog pode permanecer aberto. O projeto nao e adicionado a lista. Erro logado no console
- **Criterio de aceitacao:** Nenhum projeto fantasma deve aparecer na lista

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.project-list .section-header button').filter({ hasText: 'Novo' }).click()`
  - `page.locator('.p-dialog .form-group').first().locator('input').fill('Fazenda Serra do Cipo - Levantamento 2026')`
  - `page.locator('.p-dialog .form-group').nth(1).locator('textarea').fill('Levantamento aerofotogrametrico...')`
  - `page.locator('.p-dialog .form-group').nth(2).locator('input').fill('Serra do Cipo, MG')`
  - `page.locator('.p-dialog-footer button').filter({ hasText: 'Criar' }).click()`
- **Asserts:**
  - `await expect(page.locator('.p-dialog')).toBeVisible()` (apos passo 1)
  - `await expect(page.locator('.p-dialog')).toBeHidden()` (apos passo 5)
  - `await expect(page.locator('.project-item .project-name').filter({ hasText: 'Fazenda Serra do Cipo' })).toBeVisible()`
- **Screenshots:**
  - `screenshot_uc001_01_dialog_aberto.png`
  - `screenshot_uc001_02_formulario_preenchido.png`
  - `screenshot_uc001_03_projeto_na_lista.png`

UC-002: Buscar Projetos por Texto e Filtrar por Status

Identificacao

- **ID:** UC-002
- **Modulo:** Projetos
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Localizar rapidamente um projeto especifico na lista utilizando filtro de texto (por nome ou area de estudo) e/ou filtro por status (ativo, planejado, em andamento, concluido, arquivado).

Pre-condicoes

- Existem pelo menos 3 projetos cadastrados no sistema com diferentes nomes, areas e status
- A sidebar esquerda esta visivel

Pos-condicoes

- A lista de projetos exibe apenas os projetos que correspondem aos criterios de busca
- Os filtros permanecem ativos ate serem limpos

Elementos de UI Envolvidos

- InputText de busca global na barra superior (placeholder "Buscar...", classe `.search-input`), com pesquisa por nome, descricao ou area de estudo via endpoint autenticado.
- InputText de filtro local (placeholder "Filtrar por area...", classe `.filter-input`), aplicado aos projetos atualmente listados.
- Select de status (placeholder "Status", classe `.filter-select`, opcoes: Todos, Ativo, Planejado, Em andamento, Concluido, Arquivado).
- Lista de `.project-item` filtrada e selecionável; selecionar um resultado ativa o projeto.
- Estado de carregamento, total de correspondencias, mensagem de falha de rede e mensagem "Nenhum projeto corresponde à busca" quando uma consulta global nao encontra resultados.
- Limpar a busca global restaura a lista de projetos acessiveis ao usuario.

Fluxo Principal

Passo 1 - Verificar estado inicial da lista

- **Acao do ator:** Observa a lista de projetos na sidebar
- **Contexto visual anterior:** Sidebar esquerda visivel, todos os projetos listados, filtros em branco
- **Elemento de UI utilizado:** Container `.project-list` com `.filters` contendo `.filter-input` e `.filter-select`
- **Resposta imediata do sistema:** N/A - observacao
- **Processamento funcional:** O computed `filteredProjects` retorna todos os projetos do store pois `filterArea` esta vazio e `filterStatus` e null
- **Efeito observavel na UI:** Todos os projetos cadastrados sao exibidos na lista
- **Evidencia esperada:** Screenshot mostrando lista completa de projetos
- **Criterio de aceitacao:** O numero de `.project-item` vissiveis deve ser igual ao total de projetos no sistema
- **Seletor Playwright:** `page.locator('.project-list .project-item')`

Passo 2 - Digitar texto no filtro de area/nome

- **Acao do ator:** Digita "Serra" no campo de filtro de texto
- **Contexto visual anterior:** Lista mostra todos os projetos, campo de filtro vazio
- **Elemento de UI utilizado:** InputText PrimeVue com placeholder "Filtrar por area...", classe `.filter-input`, tamanho `small`
- **Resposta imediata do sistema:** A lista e filtrada em tempo real conforme cada caractere e digitado
- **Processamento funcional:** O v-model `filterArea` e atualizado. O computed `filteredProjects` aplica `toLowerCase().includes(q)` no `area_estudo` e `nome` de cada projeto
- **Efeito observavel na UI:** Apenas projetos cujo nome ou area contem "serra" (case-insensitive) permanecem visiveis. Projetos que nao correspondem desaparecem
- **Evidencia esperada:** Screenshot mostrando lista filtrada com apenas projetos que contem "Serra"
- **Criterio de aceitacao:** Cada `.project-item` visivel deve ter "Serra" no `.project-name` ou `.project-area`
- **Seletor Playwright:** `page.locator('.project-list .filter-input input')`

Passo 3 - Selecionar filtro de status

- **Acao do ator:** Abre o dropdown de status e seleciona "Ativo"
- **Contexto visual anterior:** Lista ja filtrada por texto "Serra", dropdown de status mostra "Status" (placeholder)
- **Elemento de UI utilizado:** Select PrimeVue com classe `.filter-select`, tamanho `small`, opcoes definidas em `statusOptions`
- **Resposta imediata do sistema:** O dropdown abre mostrando as opcoes: Todos, Ativo, Planejado, Em andamento, Concluido, Arquivado
- **Processamento funcional:** O v-model `filterStatus` e setado para `'ativo'`. O computed `filteredProjects` agora aplica dois filtros: texto E status

- **Efeito observavel na UI:** A lista mostra apenas projetos que contem "Serra" E tem status "ativo"
- **Evidencia esperada:** Screenshot mostrando lista com duplo filtro aplicado
- **Criterio de aceitacao:** Cada `.project-item` visivel deve ter badge "ativo" e conter "Serra" no nome/area
- **Seletor Playwright:** `page.locator('.project-list .filter-select')`

Passo 4 - Limpar filtro de texto

- **Acao do ator:** Apaga o texto do campo de filtro (seleciona tudo e deleta)
- **Contexto visual anterior:** Lista filtrada por "Serra" + status "ativo"
- **Elemento de UI utilizado:** `.filter-input input`
- **Resposta imediata do sistema:** A lista expande para mostrar todos os projetos com status "ativo" (independente do nome)
- **Processamento funcional:** `filterArea` e setado para string vazia, o filtro de texto nao se aplica mais
- **Efeito observavel na UI:** Mais projetos podem aparecer na lista, todos com badge "ativo"
- **Evidencia esperada:** Screenshot mostrando filtro apenas por status
- **Criterio de aceitacao:** Todos os `.project-item` vissiveis devem ter badge "ativo"
- **Seletor Playwright:** `page.locator('.project-list .filter-input input')`

Passo 5 - Selecionar "Todos" no filtro de status

- **Acao do ator:** Seleciona "Todos" no dropdown de status
- **Contexto visual anterior:** Lista filtrada apenas por status "ativo"
- **Elemento de UI utilizado:** Select PrimeVue `.filter-select`
- **Resposta imediata do sistema:** Todos os projetos voltam a aparecer na lista
- **Processamento funcional:** `filterStatus` e setado para `null`. O computed retorna `projectStore.projects` sem filtro
- **Efeito observavel na UI:** Lista completa restaurada
- **Evidencia esperada:** Screenshot mostrando todos os projetos visiveis novamente
- **Criterio de aceitacao:** Numero de `.project-item` deve ser igual ao total de projetos
- **Seletor Playwright:** `page.locator('.project-list .filter-select')`

Fluxos Alternativos

FA-1: Busca que nao retorna resultados

- **Passo divergente:** Passo 2
- **Acao do ator:** Digita "XYZNONEXISTENT" no campo de filtro
- **Processamento funcional:** `filteredProjects` retorna array vazio
- **Efeito observavel na UI:** A mensagem "Nenhum projeto encontrado" aparece na div `.empty-state`
- **Criterio de aceitacao:** Nenhum `.project-item` visivel. `.empty-state` deve conter "Nenhum projeto encontrado"
- **Seletor Playwright:** `page.locator('.project-list .empty-state')`

Fluxos de Excecao

FE-1: Projetos nao carregados (loading)

- **Condicao:** O store ainda esta carregando projetos do backend
- **Efeito observavel na UI:** O ProgressSpinner e exibido com texto "Carregando..." na div `.loading-state`
- **Criterio de aceitacao:** `.loading-state` deve estar visivel enquanto `projectStore.loading` e true

Mapeamento Playwright

- **Acoes:**

    - `page.locator('.project-list .filter-input input').fill('Serra')`
    - `page.locator('.project-list .filter-select').click()`
    - `page.locator('.p-select-option').filter({ hasText: 'Ativo' }).click()`
    - `page.locator('.project-list .filter-input input').clear()`
    - `page.locator('.project-list .filter-select').click()`
    - `page.locator('.p-select-option').filter({ hasText: 'Todos' }).click()`
- **Asserts:**
    - `await expect(page.locator('.project-item')).toHaveCount(expectedFilteredCount)`
    - `await expect(page.locator('.project-item .project-name')).toContainText(['Serra'])`
    - `await expect(page.locator('.empty-state')).toBeHidden()`
- **Screenshots:**
    - `screenshot_uc002_01_lista_completa.png`
    - `screenshot_uc002_02_filtro_texto.png`
    - `screenshot_uc002_03_filtro_texto_status.png`
    - `screenshot_uc002_04_filtro_status_apenas.png`
    - `screenshot_uc002_05_lista_restaurada.png`

UC-003: Selecionar Projeto e Visualizar Ortomapas

Identificacao

- **ID:** UC-003
- **Modulo:** Projetos
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Selecionar um projeto existente na lista para que seus ortomapas, analises, anotacoes e voos sejam carregados e exibidos na interface.

Pre-condicoes

- Existe pelo menos um projeto cadastrado com ortomapas associados
- A sidebar esquerda esta visivel

Pos-condicoes

- O projeto selecionado esta ativo no store (`projectStore.activeProject`)
- Os ortomapas do projeto sao exibidos na secao "Ortomapas" da sidebar
- Os voos, analises e anotacoes sao carregados
- O mapa pode ajustar o centro/zoom para o bbox do projeto

Elementos de UI Envolvidos

- `.project-item` na lista de projetos
- `.project-item.active` (estilo do projeto selecionado com borda verde)
- Secao "Ortomapas" com componentes `OrtomapCard`
- Secao `VoosList`
- Componente `AnalysisResults`
- MapViewer com tiles do ortomapa

Fluxo Principal

Passo 1 - Clicar em um projeto na lista

- **Acao do ator:** Clica em um `.project-item` na lista de projetos
- **Contexto visual anterior:** Lista de projetos exibida, nenhum projeto selecionado (nenhum com classe `.active`)
- **Elemento de UI utilizado:** Div `.project-item` contendo `.project-icon`, `.project-info` e `.project-badge`
- **Resposta imediata do sistema:** O item clicado recebe a classe `.active` (fundo verde claro, borda esquerda verde)
- **Processamento funcional:** Chama `selectProject(project)` que executa `projectStore.setActiveProject(project)`. Isso dispara `Promise.all([fetchOrtomapas(), fetchAnalises(), fetchAnotacoes()])` fazendo tres requisicoes GET paralelas ao backend: `/api/ortomapas?projeto_id=X`, `/api/analises?projeto_id=X`, `/api/anotacoes?projeto_id=X`. Tambem inicia o polling de tarefas a cada 5 segundos
- **Efeito observavel na UI:** (1) O `.project-item` clicado ganha estilo `.active`. (2) Pode aparecer um ProgressSpinner enquanto os dados carregam
- **Evidencia esperada:** Screenshot mostrando o projeto com estilo ativo (borda verde esquerda, fundo verde claro)
- **Criterio de aceitacao:** O `.project-item` clicado deve ter a classe CSS `active`
- **Seletor Playwright:** `page.locator('.project-item').first().click()`

Passo 2 - Verificar ortomapas carregados na sidebar

- **Acao do ator:** Observa a secao "Ortomapas" abaixo da lista de projetos
- **Contexto visual anterior:** Secao "Ortomapas" mostrando "Nenhum ortomapa carregado" ou lista vazia
- **Elemento de UI utilizado:** Secao `.sidebar-section` com titulo "Ortomapas", Badge com contagem, e componentes `OrtomapCard`
- **Resposta imediata do sistema:** Os OrtomapCards aparecem mostrando nome, tipo (badge RGB/DSM/DTM), resolucao e data
- **Processamento funcional:** `projectStore.ortomapas` foi populado pelo `fetchOrtomapas()` com dados retornados pelo endpoint GET `/api/ortomapas?projeto_id=X`
- **Efeito observavel na UI:** Cada ortomapa e exibido como um card com thumbnail (ou icone placeholder), nome, badge de tipo, resolucao em cm/px e data de voo. O Badge no titulo mostra a contagem
- **Evidencia esperada:** Screenshot da secao Ortomapas com cards preenchidos
- **Criterio de aceitacao:** O numero de `OrtomapCard` renderizados deve ser igual ao numero de ortomapas retornados pela API
- **Seletor Playwright:** `page.locator('.ortomapa-card')`

Passo 3 - Ativar visibilidade de um ortomapa no mapa

- **Acao do ator:** Clica no botao de visibilidade (icone olho) de um OrtomapCard
- **Contexto visual anterior:** OrtomapCard exibido com icone `pi-eye-slash` (invisivel)
- **Elemento de UI utilizado:** Botao `.action-btn` com icone `pi pi-eye` ou `pi pi-eye-slash` dentro de `.card-actions` do `OrtomapCard`
- **Resposta imediata do sistema:** O icone muda para `pi-eye` (ativo) e o botao ganha classe `.active`
- **Processamento funcional:** `toggleVisibility()` verifica se a layer `orto-{id}` existe em `mapStore.activeLayers`. Se nao existe, chama `mapStore.addLayer()` com tipo `ortomapa` e `sourceId`. Se existe, chama `mapStore.toggleLayer()`. O MapViewer reage ao computed `visibleOrtoLayers` e renderiza um `l-tile-layer` com URL `/api/ortomapas/{id}/tile/{z}/{x}/{y}.png`
- **Efeito observavel na UI:** O ortomapa aparece como tiles sobre o mapa base no componente MapViewer
- **Evidencia esperada:** Screenshot do mapa com tiles do ortomapa visiveis
- **Criterio de aceitacao:** Tiles do ortomapa devem ser carregadas e visiveis no mapa. O icone do botao deve ser `pi-eye`
- **Seletor Playwright:** `page.locator('.ortomapa-card .card-actions .action-btn').first().click()`

Passo 4 - Verificar voos carregados

- **Acao do ator:** Rola a sidebar para baixo ate a secao "Voos"
- **Contexto visual anterior:** Secao Voos pode estar vazia ou com mensagem "Nenhum voo registrado"
- **Elemento de UI utilizado:** Componente VoosList com `.section-header` (titulo "Voos" e Badge), `.voo-item` para cada voo
- **Resposta imediata do sistema:** N/A
- **Processamento funcional:** VoosList faz `watch(() => projectStore.activeProject, fetchVoos)` que chama GET `/api/voos?projeto_id=X`
- **Efeito observavel na UI:** Voos do projeto aparecem com data, drone, altitude e numero de fotos
- **Evidencia esperada:** Screenshot da secao Voos com itens listados
- **Criterio de aceitacao:** A secao Voos deve mostrar os voos vinculados ao projeto ativo
- **Seletor Playwright:** `page.locator('.voos-list .voo-item')`

Passo 5 - Verificar que o mapa centralizou na area do projeto

- **Acao do ator:** Observa o mapa central
- **Contexto visual anterior:** Mapa pode estar centralizado na posicao padrao (-15.78, -47.93)
- **Elemento de UI utilizado:** Componente MapViewer com `l-map`
- **Resposta imediata do sistema:** O mapa faz pan/zoom para a area do projeto
- **Processamento funcional:** O `watch` em MapViewer detecta mudanca em `projectStore.activeProject`. Se o projeto tem `bbox`, calcula o centro como `[(bbox[1]+bbox[3])/2, (bbox[0]+bbox[2])/2]` e seta zoom 14
- **Efeito observavel na UI:** O mapa mostra a regiao geografica do projeto
- **Evidencia esperada:** Screenshot do mapa centralizado na area do projeto
- **Criterio de aceitacao:** As coordenadas na barra de status devem ser compatriveis com a localizacao do projeto
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Projeto sem ortomapas

- **Condicao:** O projeto selecionado nao tem ortomapas
- **Efeito observavel na UI:** Secao "Ortomapas" mostra "Nenhum ortomapa carregado" na `.empty-state`, Badge mostra 0
- **Seletor Playwright:** `page.locator('.sidebar-section .empty-state').filter({ hasText: 'Nenhum ortomapa carregado' })`

Fluxos de Excecao

FE-1: Falha ao carregar ortomapas

- **Condicao:** Endpoint `/api/ortomapas` retorna erro
- **Processamento funcional:** `fetchOrtomapas` captura o erro e seta `this.error = e.message`
- **Efeito observavel na UI:** Lista de ortomapas permanece vazia. Erro logado no console

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.project-item').first().click()`
  - `page.locator('.ortomapa-card .card-actions .action-btn').first().click()`
- **Asserts:**
  - `await expect(page.locator('.project-item.active')).toHaveCount(1)`
  - `await expect(page.locator('.ortomapa-card')).toHaveCount(expectedCount)`

    - `await expect(page.locator('.voos-list .voo-item')).toBeVisible()`
- **Screenshots:**
    - `screenshot_uc003_01_projeto_selecionado.png`
    - `screenshot_uc003_02_ortomapas_listados.png`
    - `screenshot_uc003_03_ortomapa_no_mapa.png`
    - `screenshot_uc003_04_voos_listados.png`

UC-004: Fazer Upload de GeoTIFF e Visualizar no Mapa

Identificacao

- **ID:** UC-004
- **Modulo:** Ortomapas
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Enviar um arquivo GeoTIFF para o sistema, vincula-lo ao projeto ativo, e visualiza-lo como camada de tiles no mapa interativo Leaflet.

Pre-condicoes

- Um projeto esta selecionado (ativo) e o usuario tem permissao de edicao
- O usuario possui GeoTIFF `.tif` ou `.tiff` valido com sistema de coordenadas
- A API de upload esta acessivel

Pos-condicoes

- O arquivo GeoTIFF foi armazenado no servidor
- Um registro de ortomapa foi criado no banco de dados vinculado ao projeto
- O ortomapa aparece na lista da sidebar como OrtomapCard
- Tiles do ortomapa podem ser carregadas no mapa

Elementos de UI Envolvidos

- Botao "Importar GeoTIFF" na secao Ortomapas da sidebar
- Dialogo com tipo do raster, nome opcional, seletor de arquivo, progresso, erro e metadados retornados
- OrtomapCard na sidebar apos o upload, com thumbnail e tipo
- Camada do raster ativada automaticamente no MapViewer

Fluxo Principal

Passo 1 - Abrir o formulario de importacao

- **Acao do ator:** Clica em "Importar GeoTIFF" na secao Ortomapas.
- **Contexto visual anterior:** Projeto editavel ativo, secao Ortomapas visivel.
- **Elemento de UI utilizado:** Botao `Importar GeoTIFF` e Dialog `Importar GeoTIFF`.
- **Resposta imediata do sistema:** Dialog mostra projeto ativo, tipo, nome opcional e seletor `.tif/.tiff`.
- **Efeito observavel na UI:** O usuario consegue escolher produto e arquivo antes de enviar.
- **Evidencia esperada:** Captura do formulario de upload.
- **Criterio de aceitacao:** O dialog identifica corretamente o projeto ativo e oferece os campos previstos.

- **Seletor Playwright:** `page.getByRole('button', { name: 'Importar GeoTIFF' })`

Passo 2 - Enviar um GeoTIFF

- **Acao do ator:** Seleciona arquivo, tipo e nome opcional e clica "Enviar e adicionar ao mapa".
- **Processamento funcional:** Envia `multipart/form-data` para `/api/ortomapas/upload` com `projeto_id`, `tipo` e, quando informado, `nome`; exibe progresso.
- **Efeito observavel na UI:** Formulario informa sucesso com dimensoes, CRS e tamanho do arquivo.
- **Evidencia esperada:** Captura do resultado e resposta HTTP 201.
- **Criterio de aceitacao:** O registro fica ligado ao projeto; voo e opcional; metadados espaciais e thumbnail sao gravados.
- **Seletor Playwright:** `page.locator('#geotiff-file')` e botao `Enviar e adicionar ao mapa`

Passo 3 - Aguardar processamento do backend

- **Acao do ator:** Aguarda a conclusao do upload e processamento
- **Contexto visual anterior:** Upload em progresso
- **Elemento de UI utilizado:** Spinner ou indicador de progresso
- **Resposta imediata do sistema:** O backend recebe o arquivo e processa
- **Processamento funcional:** O backend salva o arquivo, le CRS, bbox, resolucao e bandas, gera thumbnail e registra metadados no PostgreSQL. Os tiles XYZ sao renderizados sob demanda quando o mapa requisita cada tile; nao ha piramide pre-computada neste fluxo.
- **Efeito observavel na UI:** O indicador de progresso muda para estado completo
- **Evidencia esperada:** Screenshot apos conclusao do upload
- **Criterio de aceitacao:** A API deve retornar status 200/201 com dados do ortomapa incluindo `id`, `nome`, `tipo`, `caminho_arquivo`
- **Seletor Playwright:** `page.waitForResponse(resp => resp.url().includes('/api/ortomapas/upload'))`

Passo 4 - Verificar ortomapa na lista da sidebar

- **Acao do ator:** Observa a secao Ortomapas na sidebar
- **Contexto visual anterior:** Upload concluido
- **Elemento de UI utilizado:** Componente OrtomapCard com `.card-thumbnail`, `.card-title`, `.card-meta` (Badge de tipo, resolucao)
- **Resposta imediata do sistema:** Um novo OrtomapCard aparece na lista
- **Processamento funcional:** Apos o upload, `projectStore.fetchOrtomapas()` e chamado para recarregar a lista
- **Efeito observavel na UI:** Novo card com nome do arquivo, badge de tipo (RGB, DSM, etc), resolucao e thumbnail
- **Evidencia esperada:** Screenshot da sidebar com novo OrtomapCard
- **Criterio de aceitacao:** O OrtomapCard deve exibir nome correto e tipo correto
- **Seletor Playwright:** `page.locator('.ortomapa-card').last()`

Passo 5 - Mostrar o ortomapa

- **Acao do sistema:** Apos upload bem-sucedido, adiciona o ortomapa como camada visivel no mapa.
- **Processamento funcional:** `mapStore.addLayer({ id: 'orto-{id}', name, type: 'ortomapa', sourceId: id })`. MapViewer solicita tiles autenticadas em `/api/ortomapas/{id}/tile/{z}/{x}/{y}.png`.
- **Efeito observavel na UI:** O card aparece com thumbnail e o raster e sobreposto ao mapa base; controles de opacidade continuam disponiveis.
- **Evidencia esperada:** Captura do mapa, card, thumbnail e respostas de tiles.
- **Criterio de aceitacao:** Tiles e thumbnail respondem HTTP 200 e o mapa enquadra a extensao do raster.

Passo 6 - Ajustar opacidade da camada

- **Acao do ator:** Usa o controle de opacidade no mapa (`.layer-controls`)
- **Contexto visual anterior:** Ortomapa visivel no mapa, controles de camada aparecem no canto superior direito
- **Elemento de UI utilizado:** Slider `input[type=range]` em `.layer-control-item` dentro de `.layer-controls`
- **Resposta imediata do sistema:** A opacidade da camada muda em tempo real
- **Processamento funcional:** `mapStore.setLayerOpacity(layer.id, value)`. A propriedade `:opacity` do `l-tile-layer` e atualizada reativamente
- **Efeito observavel na UI:** O ortomapa fica mais transparente ou opaco conforme o slider
- **Evidencia esperada:** Screenshot com ortomapa em opacidade intermediaria (~50%)
- **Criterio de aceitacao:** A opacidade visual deve corresponder ao valor do slider
- **Seletor Playwright:** `page.locator('.layer-controls .layer-control-item input[type="range"]').first()`

Fluxos Alternativos

FA-1: Upload de DSM/DTM

- **Passo divergente:** Passo 1
- **Diferenca:** O parametro `tipo` e setado para 'DSM' ou 'DTM' em vez de 'RGB'
- **Efeito observavel na UI:** O OrtomapCard mostra badge "DSM" com severity `warn` (amarelo)

Fluxos de Excecao

FE-1: Arquivo invalido (nao e GeoTIFF)

- **Condicao:** O usuario envia um arquivo que nao e GeoTIFF valido
- **Processamento funcional:** O backend rejeita o arquivo e retorna erro 400/422
- **Efeito observavel na UI:** Mensagem de erro e exibida ao usuario
- **Criterio de aceitacao:** Nenhum ortomapa deve ser criado. Mensagem de erro deve ser clara

FE-2: Timeout de upload (arquivo muito grande)

- **Condicao:** O upload excede 300 segundos
- **Processamento funcional:** Axios lanca erro de timeout
- **Efeito observavel na UI:** Mensagem de erro de timeout. Upload interrompido

Mapeamento Playwright

- **Acoes:**
  - `page.locator('input[type="file"]').setInputFiles('/path/to/ortomapa.tif')`
  - `page.waitForResponse(resp => resp.url().includes('/api/ortomapas/upload') && resp.status() === 200)`
  - `page.locator('.ortomapa-card .card-actions .action-btn').first().click()`
  - `page.locator('.layer-controls input[type="range"]').first().fill('50')`
- **Asserts:**
  - `await expect(page.locator('.ortomapa-card')).toHaveCount(previousCount + 1)`
  - `await expect(page.locator('.ortomapa-card .card-title').last()).toContainText('nome_do_arquivo')`
- **Screenshots:**
  - `screenshot_uc004_01_upload_progresso.png`
  - `screenshot_uc004_02_ortomapa_na_sidebar.png`
  - `screenshot_uc004_03_ortomapa_no_mapa.png`
  - `screenshot_uc004_04_opacidade_ajustada.png`

UC-005: Calcular Indice de Vegetacao VARI

Identificacao

- **ID:** UC-005
- **Modulo:** Analise
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Calcular um indice de vegetacao visivel a partir de um ortomapa RGB, opcionalmente limitado por AOI, publicar o raster no mapa e consultar min, max, media e pixels validos. O fluxo nao calcula NDVI sem banda NIR.

Pre-condicoes

- Um projeto esta ativo com pelo menos um ortomapa do tipo RGB
- A sidebar direita esta visivel com o ToolsPanel
- A aba "Veg" esta disponivel no ToolsPanel

Pos-condicoes

- Uma nova analise de vegetacao foi criada no banco de dados
- O resultado raster do VARI esta disponivel como camada no mapa
- As estatisticas (min, max, media e pixels validos) sao exibidas no painel

Elementos de UI Envolvidos

- ToolsPanel na sidebar direita
- Tab "Veg" (Tabs PrimeVue, valor `vegetacao`)
- Select "Ortomapa" (dropdown de ortomapas do projeto)
- Select "Indice de Vegetacao" (opcoes: VARI, TGI, ExG, GLI)
- Button "Calcular Indice" (icon `pi pi-play`)
- Div `.result-stats` com estatisticas
- MapViewer com camada de resultado

Fluxo Principal

Passo 1 - Navegar para a aba Vegetacao no ToolsPanel

- **Acao do ator:** Clica na aba "Veg" no ToolsPanel da sidebar direita
- **Contexto visual anterior:** Sidebar direita visivel, ToolsPanel mostrando abas. Pode estar em qualquer aba
- **Elemento de UI utilizado:** Tab PrimeVue com valor `vegetacao`, conteudo `<i class="pi pi-sun"></i> Veg`
- **Resposta imediata do sistema:** O conteudo da aba de vegetacao e exibido
- **Processamento funcional:** `activeTab` e setado para `'vegetacao'`
- **Efeito observavel na UI:** O formulario de analise de vegetacao aparece com Select de ortomapa, Select de indice e botao "Calcular Indice"
- **Evidencia esperada:** Screenshot da aba vegetacao aberta
- **Criterio de aceitacao:** A aba "Veg" deve estar ativa e o formulario de vegetacao visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="vegetacao"]')`

Passo 2 - Selecionar o ortomapa

- **Acao do ator:** Abre o dropdown "Ortomapa" e seleciona um ortomapa RGB
- **Contexto visual anterior:** Dropdown mostrando "Selecionar..." (placeholder)
- **Elemento de UI utilizado:** Select PrimeVue com v-model `veg.ortomapaId`, opcoes vindas de `rgbOptions`; DSM e DTM nao aparecem.
- **Resposta imediata do sistema:** O dropdown fecha e mostra o nome do ortomapa selecionado
- **Processamento funcional:** `veg.ortomapaId` e setado para o `id` do ortomapa selecionado
- **Efeito observavel na UI:** O nome do ortomapa aparece no dropdown
- **Evidencia esperada:** Screenshot com ortomapa selecionado no dropdown
- **Criterio de aceitacao:** `veg.ortomapaId` deve ser um ID valido
- **Seletor Playwright:** `page.locator('.tools-panel [value="vegetacao"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar o indice VARI

- **Acao do ator:** Verifica que "VARI" esta selecionado (e o padrao) ou seleciona-o
- **Contexto visual anterior:** Dropdown de indice mostrando "VARI - Visible Atmospherically Resistant Index"
- **Elemento de UI utilizado:** Select PrimeVue com v-model `veg.index`, opcoes: VARI, TGI, ExG, GLI
- **Resposta imediata do sistema:** N/A - VARI ja e o valor padrao
- **Processamento funcional:** `veg.index` permanece como `'VARI'`
- **Efeito observavel na UI:** Dropdown mostra "VARI - Visible Atmospherically Resistant Index"
- **Evidencia esperada:** Screenshot confirmando selecao do VARI
- **Criterio de aceitacao:** O valor selecionado deve ser 'VARI'
- **Seletor Playwright:** `page.locator('.tools-panel [value="vegetacao"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Clicar em "Calcular Indice"

- **Acao do ator:** Clica no botao "Calcular Indice"
- **Contexto visual anterior:** Ortomapa e indice selecionados, botao habilitado
- **Elemento de UI utilizado:** Button PrimeVue com label "Calcular Indice", icon `pi pi-play`, classe `w-full`
- **Resposta imediata do sistema:** O botao entra em estado loading (spinner + desabilitado)
- **Processamento funcional:** (1) Valida a fonte RGB e a AOI opcional. (2) Seta `veg.loading = true`. (3) Chama `runTool('vegetation', {input_path, ortomapa_id, projeto_id, aoi, output_name, index_name})`, via POST `/api/tools/vegetation`. (4) O backend preserva nodata, calcula e recorta o indice. (5) Publica GeoTIFF, estatisticas, estilo e proveniencia em `analises`. (6) A interface adiciona a camada persistente e atualiza o catalogo.
- **Efeito observavel na UI:** O botao mostra spinner enquanto processa
- **Evidencia esperada:** Screenshot do botao em estado loading
- **Criterio de aceitacao:** O botao deve estar desabilitado com icone de spinner durante o processamento
- **Seletor Playwright:** `page.locator('.tools-panel [value="vegetacao"] button').filter({ hasText: 'Calcular Indice' })`

Passo 5 - Visualizar resultados estatisticos

- **Acao do ator:** Observa os resultados que aparecem abaixo do botao
- **Contexto visual anterior:** Botao em estado loading
- **Elemento de UI utilizado:** Div `.result-stats` com titulo "Resultado" e quatro linhas `.stat-row`: Min, Max, Media, Pixels validos
- **Resposta imediata do sistema:** Os valores estatisticos aparecem formatados com 3 casas decimais
- **Processamento funcional:** `veg.result` recebe a resposta; `addResultLayer()` usa a analise publicada e `fetchAnalises()` recarrega o catalogo.
- **Efeito observavel na UI:** Bloco mostra min, max, media, pixels validos e o aviso de que o indice usa RGB visivel, sem NIR.

- **Evidencia esperada:** Screenshot dos resultados estatisticos
- **Criterio de aceitacao:** Min, max e media sao numeros validos; pixels validos e inteiro positivo; Min <= Media <= Max.
- **Seletor Playwright:** `page.locator('.tools-panel .result-stats')`

Passo 6 - Verificar camada de resultado no mapa

- **Acao do ator:** Observa o mapa para verificar a camada VARI
- **Contexto visual anterior:** Mapa mostrando ortomapa base ou tiles OSM
- **Elemento de UI utilizado:** MapViewer com camada de analise adicionada via `mapStore.addLayer()`
- **Resposta imediata do sistema:** Uma camada colorida representando o VARI aparece no mapa
- **Processamento funcional:** `addResultLayer('VARI', res.data)` cria layer no mapStore. Se `data.tile_url` existe, adiciona tambem uma tile layer raster. O MapViewer renderiza via `l-tile-layer` ou `l-geo-json`
- **Efeito observavel na UI:** Camada colorizada do VARI sobreposta ao mapa, com gradiente de cores representando valores de vegetacao
- **Evidencia esperada:** Screenshot do mapa com camada VARI visivel
- **Criterio de aceitacao:** A camada VARI deve estar visivel e cobrir a mesma area do ortomapa original
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Selecionar outro indice (TGI, ExG, GLI)

- **Passo divergente:** Passo 3
- **Acao do ator:** Seleciona "TGI" em vez de "VARI"
- **Processamento funcional:** `veg.index` e setado para `'TGI'`. O backend calcula TGI em vez de VARI
- **Efeito observavel na UI:** Os resultados estatisticos correspondem ao indice TGI. A camada no mapa mostra valores TGI

Fluxos de Excecao

FE-1: Nenhum ortomapa selecionado

- **Condicao:** `veg.ortomapaId` e null
- **Acao do ator:** Clica em "Calcular Indice" sem selecionar ortomapa
- **Processamento funcional:** `if (!veg.ortomapaId) return` - retorno silencioso
- **Efeito observavel na UI:** Nada acontece, botao nao entra em loading

FE-2: Erro no processamento do backend

- **Condicao:** O backend retorna erro (ex: ortomapa corrompido, bandas insuficientes)
- **Processamento funcional:** O catch captura o erro, `console.error(e)`. `veg.loading` e setado para false no finally
- **Efeito observavel na UI:** O botao volta ao estado normal. Nenhum resultado e exibido. Erro logado no console

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="vegetacao"]').click()`
  - `page.locator('.tools-panel [value="vegetacao"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="vegetacao"] button').filter({ hasText: 'Calcular Indice' }).click()`
  - `page.waitForSelector('.tools-panel .result-stats')`

- **Asserts:**
  - `await expect(page.locator('.tools-panel .result-stats')).toBeVisible()`
  - `await expect(page.locator('.result-stats .stat-row')).toHaveCount(4)`
  - `await expect(page.locator('.result-stats .stat-row').first()).toContainText('Min')`
- **Screenshots:**
  - `screenshot_uc005_01_aba_vegetacao.png`
  - `screenshot_uc005_02_ortomapa_selecionado.png`
  - `screenshot_uc005_03_loading.png`
  - `screenshot_uc005_04_resultados_estatisticos.png`
  - `screenshot_uc005_05_camada_mapa.png`

UC-006: Calcular Declividade (Slope) a partir de DSM

Identificacao

- **ID:** UC-006
- **Modulo:** Analise
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Gerar um mapa de declividade (slope) a partir de um Modelo Digital de Superficie (DSM), visualizando a inclinacao do terreno em graus como camada raster no mapa.

Pre-condicoes

- Um projeto esta ativo com pelo menos um ortomapa do tipo DSM ou DTM
- A sidebar direita esta visivel com o ToolsPanel

Pos-condicoes

- Uma camada de declividade foi gerada e adicionada ao mapa
- Uma analise foi registrada no banco de dados
- A camada mostra gradiente de cores representando inclinacao

Elementos de UI Envolvidos

- Tab "Ter" (terreno) no ToolsPanel
- Select "Modelo (DSM/DTM)" filtrado por tipo DSM/DTM
- Select "Analise" com opcoes: Declividade, Aspecto, Curvas de Nivel, Sombreamento
- Button "Gerar Analise" (icon `pi pi-play`)

Fluxo Principal

Passo 1 - Navegar para a aba Terreno

- **Acao do ator:** Clica na aba "Ter" no ToolsPanel
- **Contexto visual anterior:** ToolsPanel visivel, pode estar em qualquer outra aba
- **Elemento de UI utilizado:** Tab PrimeVue com valor `terreno`, conteudo `<i class="pi pi-chart-line"></i> Ter`
- **Resposta imediata do sistema:** Painel de terreno exibido com campos de selecao
- **Processamento funcional:** `activeTab` setado para `'terreno'`

- **Efeito observavel na UI:** Formulario mostra Select de modelo DSM/DTM, Select de tipo de analise e botao "Gerar Analise"
- **Evidencia esperada:** Screenshot da aba terreno aberta
- **Criterio de aceitacao:** Aba terreno deve estar ativa e formulario visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="terreno"]')`

Passo 2 - Selecionar o modelo DSM

- **Acao do ator:** Abre o dropdown "Modelo (DSM/DTM)" e seleciona um DSM
- **Contexto visual anterior:** Dropdown mostrando "Selecionar..." (placeholder)
- **Elemento de UI utilizado:** Select PrimeVue com v-model `terreno.ortomapaId`, opcoes de `dsmDtmOptions` (filtra `ortomapas` com tipo DSM ou DTM, mostra `nome (tipo)`)
- **Resposta imediata do sistema:** Dropdown fecha e mostra "NomeDoOrtomapa (DSM)"
- **Processamento funcional:** `terreno.ortomapaId` e setado para o ID do DSM selecionado
- **Efeito observavel na UI:** Nome do modelo selecionado aparece no dropdown
- **Evidencia esperada:** Screenshot com DSM selecionado
- **Criterio de aceitacao:** O dropdown deve mostrar apenas ortomapas de tipo DSM ou DTM
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar tipo "Declividade (Slope)"

- **Acao do ator:** Verifica que "Declividade (Slope)" esta selecionado (padrao) ou seleciona
- **Contexto visual anterior:** Dropdown de analise mostrando "Declividade (Slope)"
- **Elemento de UI utilizado:** Select PrimeVue com v-model `terreno.tipo`, opcoes `terrenoTypes`. Valor padrao: `'slope'`
- **Resposta imediata do sistema:** N/A - ja e o valor padrao
- **Processamento funcional:** `terreno.tipo` permanece como `'slope'`
- **Efeito observavel na UI:** Dropdown mostra "Declividade (Slope)"
- **Evidencia esperada:** Screenshot confirmando selecao
- **Criterio de aceitacao:** Valor selecionado deve ser 'slope'. Campo de intervalo nao deve aparecer (so aparece para contours)
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Clicar em "Gerar Analise"

- **Acao do ator:** Clica no botao "Gerar Analise"
- **Contexto visual anterior:** DSM selecionado, tipo Slope selecionado
- **Elemento de UI utilizado:** Button PrimeVue com label "Gerar Analise", icon `pi pi-play`, loading vinculado a `terreno.loading`
- **Resposta imediata do sistema:** Botao entra em estado loading
- **Processamento funcional:** (1) Valida `terreno.ortomapaId`. (2) Seta `terreno.loading = true`. (3) Chama `runTool('terreno', { ortomapa_id, tipo: 'slope', intervalo: 5 })` via POST `/api/tools/terreno`. (4) Backend calcula gradiente de elevacao usando numpy/GDAL. (5) Gera raster de resultado. (6) Retorna camada e metadados
- **Efeito observavel na UI:** Botao com spinner durante processamento
- **Evidencia esperada:** Screenshot do botao em loading
- **Criterio de aceitacao:** Requisicao deve enviar parametros corretos: `ortomapa_id`, `tipo: 'slope'`
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' })`

Passo 5 - Verificar resultado no mapa

- **Acao do ator:** Observa a camada de slope adicionada ao mapa
- **Contexto visual anterior:** Mapa com tiles base e possivelmente ortomapa
- **Elemento de UI utilizado:** MapViewer com nova camada de analise
- **Resposta imediata do sistema:** Camada de declividade aparece no mapa
- **Processamento funcional:** `addResultLayer('slope', res.data)` adiciona layer ao mapStore. `projectStore.fetchAnalises()` atualiza lista de analises
- **Efeito observavel na UI:** Mapa mostra camada colorizada com gradiente representando declividade (verde = plano, vermelho = inclinado)
- **Evidencia esperada:** Screenshot do mapa com camada de slope
- **Criterio de aceitacao:** A camada deve cobrir a mesma extensao do DSM original
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Selecionar DTM em vez de DSM

- **Passo divergente:** Passo 2
- **Diferenca:** O usuario seleciona um DTM. O calculo de slope funciona da mesma forma, porem os resultados refletem o terreno real (sem construcoes/vegetacao)

Fluxos de Excecao

FE-1: DSM com dados NoData excessivos

- **Condicao:** O DSM tem grandes areas sem dados
- **Processamento funcional:** Backend pode retornar erro ou resultado parcial
- **Efeito observavel na UI:** Camada de resultado pode ter areas em branco/transparente

FE-2: Nenhum modelo selecionado

- **Condicao:** `terreno.ortomapaId` e null
- **Processamento funcional:** `if (!terreno.ortomapaId) return` - retorno silencioso
- **Efeito observavel na UI:** Nada acontece

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="terreno"]').click()`
  - `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' }).click()`
  - `page.waitForResponse(resp => resp.url().includes('/api/tools/terreno'))`
- **Asserts:**
  - `await expect(page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' })).not.toHaveAttribute('disabled', '')`
- **Screenshots:**
  - `screenshot_uc006_01_aba_terreno.png`
  - `screenshot_uc006_02_dsm_selecionado.png`
  - `screenshot_uc006_03_loading.png`
  - `screenshot_uc006_04_resultado_mapa.png`

UC-007: Gerar Curvas de Nivel a partir de DSM

Identificacao

- **ID:** UC-007
- **Modulo:** Analise
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Gerar linhas de curvas de nivel (isolinhas de elevacao) a partir de um DSM com intervalo configuravel pelo usuario, e visualizar como camada vetorial GeoJSON no mapa.

Pre-condicoes

- Projeto ativo com pelo menos um ortomapa do tipo DSM ou DTM
- Sidebar direita visivel

Pos-condicoes

- Curvas de nivel geradas como GeoJSON
- Camada vetorial adicionada ao mapa
- Analise registrada no banco

Elementos de UI Envolvidos

- Tab "Ter" no ToolsPanel
- Select "Modelo (DSM/DTM)"
- Select "Analise" com opcao "Curvas de Nivel (Contours)"
- InputText "Intervalo (m)" (aparece condicionalmente quando tipo e `contours`)
- Button "Gerar Analise"
- MapViewer com GeoJSON de curvas

Fluxo Principal

Passo 1 - Navegar para aba Terreno

- **Acao do ator:** Clica na aba "Ter"
- **Contexto visual anterior:** ToolsPanel em qualquer aba
- **Elemento de UI utilizado:** Tab PrimeVue com valor `terreno`
- **Resposta imediata do sistema:** Formulario de terreno exibido
- **Processamento funcional:** `activeTab = 'terreno'`
- **Efeito observavel na UI:** Aba terreno ativa
- **Evidencia esperada:** Screenshot da aba terreno
- **Criterio de aceitacao:** Aba terreno ativa
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="terreno"]')`

Passo 2 - Selecionar modelo DSM

- **Acao do ator:** Seleciona um DSM no dropdown
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select PrimeVue `terreno.ortomapaId`

- **Resposta imediata do sistema:** DSM selecionado exibido no dropdown
- **Processamento funcional:** `terreno.ortomapaId` setado
- **Efeito observavel na UI:** Nome do DSM no dropdown
- **Evidencia esperada:** Screenshot com DSM selecionado
- **Criterio de aceitacao:** ID valido selecionado
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar "Curvas de Nivel (Contours)"

- **Acao do ator:** Muda o dropdown de analise para "Curvas de Nivel (Contours)"
- **Contexto visual anterior:** Dropdown mostrando "Declividade (Slope)" (padrao)
- **Elemento de UI utilizado:** Select PrimeVue com v-model `terreno.tipo`, seleciona valor `'contours'`
- **Resposta imediata do sistema:** O dropdown fecha e um novo campo "Intervalo (m)" aparece abaixo condicionalmente (diretiva `v-if="terreno.tipo === 'contours'"`)
- **Processamento funcional:** `terreno.tipo = 'contours'`. O template condicional renderiza o InputText de intervalo
- **Efeito observavel na UI:** Campo "Intervalo (m)" aparece com valor padrao 5
- **Evidencia esperada:** Screenshot mostrando campo de intervalo visivel
- **Criterio de aceitacao:** O campo de intervalo deve estar visivel e conter o valor 5
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Configurar intervalo de curvas

- **Acao do ator:** Altera o intervalo de 5 para 10 metros
- **Contexto visual anterior:** Campo "Intervalo (m)" mostrando 5
- **Elemento de UI utilizado:** InputText PrimeVue com v-model.number `terreno.interval`, type `number`, classe `w-full`
- **Resposta imediata do sistema:** Valor muda para 10
- **Processamento funcional:** `terreno.interval = 10`
- **Efeito observavel na UI:** Campo mostra 10
- **Evidencia esperada:** Screenshot com intervalo configurado
- **Criterio de aceitacao:** O valor do campo deve ser 10
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] input[type="number"]')`

Passo 5 - Clicar em "Gerar Analise"

- **Acao do ator:** Clica no botao "Gerar Analise"
- **Contexto visual anterior:** Formulario preenchido com DSM, contours e intervalo 10
- **Elemento de UI utilizado:** Button "Gerar Analise"
- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** POST `/api/tools/terreno` com `{ortomapa_id, tipo: 'contours', intervalo: 10}`. Backend usa GDAL contour ou rasterio para gerar isolinhas. Retorna GeoJSON com LineStrings representando cada curva de nivel
- **Efeito observavel na UI:** Botao com spinner
- **Evidencia esperada:** Screenshot do loading
- **Criterio de aceitacao:** Requisicao enviada com parametros corretos
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' })`

Passo 6 - Verificar curvas de nivel no mapa

- **Acao do ator:** Observa o mapa
- **Contexto visual anterior:** Mapa com camadas anteriores
- **Elemento de UI utilizado:** MapViewer com `l-geo-json` renderizando as curvas
- **Resposta imediata do sistema:** Linhas de curvas de nivel aparecem sobre o mapa
- **Processamento funcional:** `addResultLayer('contours', res.data)` adiciona GeoJSON ao mapStore. MapViewer renderiza via `visibleAnalysisLayers` e `l-geo-json`
- **Efeito observavel na UI:** Curvas de nivel como linhas coloridas no mapa, espacadas a cada 10m de elevacao
- **Evidencia esperada:** Screenshot do mapa com curvas de nivel
- **Criterio de aceitacao:** Linhas devem ser visiveis e formar padrao topografico coerente
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Intervalo muito pequeno

- **Passo divergente:** Passo 4
- **Acao:** Usuario seta intervalo para 0.5m
- **Efeito:** Muitas curvas de nivel geradas, pode ser lento. O backend processa e retorna mais features

Fluxos de Excecao

FE-1: DSM plano (sem variacao de elevacao)

- **Condicao:** O DSM tem variacao minima de elevacao
- **Efeito:** Poucas ou nenhuma curva gerada. GeoJSON retornado pode ter poucos features

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="terreno"]').click()`
  - `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="terreno"] .form-group').nth(1).locator('.p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'Curvas de Nivel' }).click()`
  - `page.locator('.tools-panel [value="terreno"] input[type="number"]').fill('10')`
  - `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' }).click()`
- **Asserts:**
  - `await expect(page.locator('.tools-panel [value="terreno"] input[type="number"]')).toBeVisible()`
- **Screenshots:**
  - `screenshot_uc007_01_aba_terreno.png`
  - `screenshot_uc007_02_contours_selecionado.png`
  - `screenshot_uc007_03_intervalo_configurado.png`
  - `screenshot_uc007_04_curvas_no_mapa.png`

UC-008: Gerar Hillshade a partir de DSM

Identificacao

- **ID:** UC-008
- **Modulo:** Analise

- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Gerar uma visualizacao de sombreamento do relevo (hillshade) a partir de um DSM, permitindo ao usuario compreender a topografia tridimensional da area mapeada.

Pre-condicoes

- Projeto ativo com ortomapa tipo DSM ou DTM
- Sidebar direita visivel

Pos-condicoes

- Camada de hillshade gerada e adicionada ao mapa
- Analise registrada no banco

Elementos de UI Envolvidos

- Tab "Ter" no ToolsPanel
- Select de modelo DSM/DTM
- Select de analise (opcao "Sombreamento (Hillshade)")
- Button "Gerar Analise"

Fluxo Principal

Passo 1 - Navegar para aba Terreno

- **Acao do ator:** Clica na aba "Ter"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `terreno`
- **Resposta imediata do sistema:** Formulario de terreno exibido
- **Processamento funcional:** `activeTab = 'terreno'`
- **Efeito observavel na UI:** Aba terreno ativa
- **Evidencia esperada:** Screenshot aba terreno
- **Criterio de aceitacao:** Formulario visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="terreno"]')`

Passo 2 - Selecionar modelo DSM

- **Acao do ator:** Seleciona DSM no dropdown
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select `terreno.ortomapaId` com opcoes `dsmDtmOptions`
- **Resposta imediata do sistema:** DSM selecionado
- **Processamento funcional:** `terreno.ortomapaId` setado
- **Efeito observavel na UI:** Nome do DSM no dropdown
- **Evidencia esperada:** Screenshot DSM selecionado
- **Criterio de aceitacao:** ID valido
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar "Sombreamento (Hillshade)"

- **Acao do ator:** Muda dropdown de analise para "Sombreamento (Hillshade)"
- **Contexto visual anterior:** Dropdown em "Declividade (Slope)"
- **Elemento de UI utilizado:** Select `terreno.tipo`, seleciona valor `'hillshade'`
- **Resposta imediata do sistema:** Dropdown fecha mostrando "Sombreamento (Hillshade)". Nenhum campo extra condicional aparece (campo de intervalo nao aparece pois e so para contours)
- **Processamento funcional:** `terreno.tipo = 'hillshade'`
- **Efeito observavel na UI:** Dropdown mostra hillshade
- **Evidencia esperada:** Screenshot com hillshade selecionado
- **Criterio de aceitacao:** Tipo selecionado deve ser 'hillshade'
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Clicar em "Gerar Analise"

- **Acao do ator:** Clica no botao
- **Contexto visual anterior:** Formulario com DSM e hillshade selecionados
- **Elemento de UI utilizado:** Button "Gerar Analise"
- **Resposta imediata do sistema:** Loading state
- **Processamento funcional:** POST `/api/tools/terreno` com `{ortomapa_id, tipo: 'hillshade', intervalo: 5}`. Backend usa GDAL hillshade ou calculo manual com azimute e altitude solar padrao (315 e 45 graus). Gera raster de tons de cinza simulando iluminacao
- **Efeito observavel na UI:** Spinner no botao
- **Evidencia esperada:** Screenshot loading
- **Criterio de aceitacao:** Requisicao enviada corretamente
- **Seletor Playwright:** `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' })`

Passo 5 - Verificar hillshade no mapa

- **Acao do ator:** Observa o mapa
- **Contexto visual anterior:** Mapa com camadas existentes
- **Elemento de UI utilizado:** MapViewer
- **Resposta imediata do sistema:** Camada de hillshade aparece
- **Processamento funcional:** `addResultLayer('hillshade', res.data)` adiciona ao mapa
- **Efeito observavel na UI:** Imagem em tons de cinza mostrando relevo sombreado, como se iluminado por uma fonte de luz lateral. Encostas iluminadas em branco, sombras em preto
- **Evidencia esperada:** Screenshot do hillshade no mapa
- **Criterio de aceitacao:** Hillshade deve ser visivel e mostrar relevo coerente
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Gerar hillshade via AnalysisForm com azimute customizado

- **Alternativa:** Usar o AnalysisForm modal que permite configurar azimute (315) e altitude solar (45)
- **Diferenca:** Mais controle sobre parametros de iluminacao

Fluxos de Excecao

FE-1: DSM corrompido

- **Condicao:** O arquivo DSM nao pode ser lido
- **Processamento funcional:** Backend retorna erro 500

- **Efeito observavel na UI:** Botao volta ao normal, nenhuma camada adicionada

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="terreno"]').click()`
  - `page.locator('.tools-panel [value="terreno"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="terreno"] .form-group').nth(1).locator('.p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'Sombreamento' }).click()`
  - `page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' }).click()`
- **Asserts:**
  - `await expect(page.locator('.tools-panel [value="terreno"] button').filter({ hasText: 'Gerar Analise' })).toBeEnabled()`
- **Screenshots:**
  - `screenshot_uc008_01_hillshade_selecionado.png`
  - `screenshot_uc008_02_loading.png`
  - `screenshot_uc008_03_hillshade_no_mapa.png`

UC-009: Detectar Mudancas Entre Dois Ortomapas

Identificacao

- **ID:** UC-009
- **Modulo:** Analise
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Comparar dois ortomapas de datas diferentes para detectar areas onde ocorreram mudancas significativas (desmatamento, construcao, erosao, etc), configurando o limiar de deteccao e visualizando o resultado no mapa.

Pre-condicoes

- Projeto ativo com pelo menos dois ortomapas RGB
- Sidebar direita visivel

Pos-condicoes

- Mapa de mudancas gerado como camada raster/vetorial
- Areas de mudanca destacadas no mapa
- Analise registrada

Elementos de UI Envolvidos

- Tab "Mud" (mudancas) no ToolsPanel
- Select "Ortomapa Antes"
- Select "Ortomapa Depois"
- Slider "Limiar" (range 0-1, step 0.01, padrao 0.3)
- Label exibindo valor do limiar

- Button "Detectar Mudancas"

Fluxo Principal

Passo 1 - Navegar para aba Mudancas

- **Acao do ator:** Clica na aba "Mud"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `mudancas`, conteudo `<i class="pi pi-arrow-right-arrow-left"></i> Mud`
- **Resposta imediata do sistema:** Formulario de deteccao de mudancas
- **Processamento funcional:** `activeTab = 'mudancas'`
- **Efeito observavel na UI:** Formulario com dois selects de ortomapa, slider de limiar e botao
- **Evidencia esperada:** Screenshot aba mudancas
- **Criterio de aceitacao:** Formulario visivel com todos os campos
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="mudancas"]')`

Passo 2 - Selecionar ortomapa "Antes"

- **Acao do ator:** Abre dropdown "Ortomapa Antes" e seleciona o ortomapa mais antigo
- **Contexto visual anterior:** Dropdown com placeholder "Selecionar..."
- **Elemento de UI utilizado:** Select PrimeVue com v-model `mudanca.antesId`, opcoes `ortoOptions`
- **Resposta imediata do sistema:** Nome do ortomapa selecionado exibido
- **Processamento funcional:** `mudanca.antesId` setado para o ID
- **Efeito observavel na UI:** Dropdown mostra nome do ortomapa
- **Evidencia esperada:** Screenshot com "antes" selecionado
- **Criterio de aceitacao:** ID valido selecionado
- **Seletor Playwright:** `page.locator('.tools-panel [value="mudancas"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar ortomapa "Depois"

- **Acao do ator:** Abre dropdown "Ortomapa Depois" e seleciona o ortomapa mais recente
- **Contexto visual anterior:** Dropdown "Depois" com placeholder
- **Elemento de UI utilizado:** Select PrimeVue com v-model `mudanca.depoisId`
- **Resposta imediata do sistema:** Nome do segundo ortomapa exibido
- **Processamento funcional:** `mudanca.depoisId` setado
- **Efeito observavel na UI:** Ambos os dropdowns preenchidos
- **Evidencia esperada:** Screenshot com ambos selecionados
- **Criterio de aceitacao:** Antes e depois devem ser ortomapas diferentes
- **Seletor Playwright:** `page.locator('.tools-panel [value="mudancas"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Ajustar o limiar de deteccao

- **Acao do ator:** Arrasta o slider de limiar de 0.30 para 0.20
- **Contexto visual anterior:** Label mostra "Limiar: 0.30", slider no centro-esquerda
- **Elemento de UI utilizado:** Input `type="range"` com v-model.number `mudanca.threshold`, min 0, max 1, step 0.01, classe `slider`
- **Resposta imediata do sistema:** O label atualiza em tempo real para "Limiar: 0.20"
- **Processamento funcional:** `mudanca.threshold` e atualizado reativamente. O label usa template `{{ mudanca.threshold.toFixed(2) }}`

- **Efeito observavel na UI:** Label mostra "Limiar: 0.20", slider posicionado em ~20%
- **Evidencia esperada:** Screenshot com limiar ajustado
- **Criterio de aceitacao:** O valor exibido deve corresponder a posicao do slider
- **Seletor Playwright:** `page.locator('.tools-panel [value="mudancas"] input[type="range"]')`

Passo 5 - Clicar em "Detectar Mudancas"

- **Acao do ator:** Clica no botao
- **Contexto visual anterior:** Ambos ortomapas selecionados, limiar em 0.20
- **Elemento de UI utilizado:** Button "Detectar Mudancas" com icon `pi pi-play`
- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** (1) Valida `antesId` e `depoisId`. (2) POST `/api/tools/mudancas` com `{antes_id, depois_id, threshold: 0.20}`. (3) Backend: carrega ambos rasters, calcula diferenca normalizada, aplica limiar, gera mapa binario de mudancas. (4) Retorna resultado com areas de mudanca
- **Efeito observavel na UI:** Spinner no botao
- **Evidencia esperada:** Screenshot loading
- **Criterio de aceitacao:** Requisicao com parametros corretos
- **Seletor Playwright:** `page.locator('.tools-panel [value="mudancas"] button').filter({ hasText: 'Detectar Mudancas' })`

Passo 6 - Visualizar resultado no mapa

- **Acao do ator:** Observa camada de mudancas no mapa
- **Contexto visual anterior:** Mapa com ortomapas base
- **Elemento de UI utilizado:** MapViewer com nova camada
- **Resposta imediata do sistema:** Areas de mudanca destacadas em cor contrastante
- **Processamento funcional:** `addResultLayer('Deteccao de Mudancas', res.data)` adiciona camada
- **Efeito observavel na UI:** Regioes que mudaram sao destacadas (tipicamente em vermelho/laranja) sobre o mapa
- **Evidencia esperada:** Screenshot do mapa com mudancas
- **Criterio de aceitacao:** Areas de mudanca devem ser visiveis e espacialmente coerentes
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Limiar alto (0.80) - pouca deteccao

- **Passo divergente:** Passo 4
- **Acao:** Usuario seta limiar para 0.80
- **Efeito:** Apenas mudancas muito intensas sao detectadas, resultando em menos areas destacadas

Fluxos de Excecao

FE-1: Ortomapas com resolucoes diferentes

- **Condicao:** Os dois ortomapas tem resolucoes ou extensoes diferentes
- **Processamento funcional:** Backend pode reamostrar automaticamente ou retornar erro
- **Efeito observavel na UI:** Resultado pode ser parcial ou erro exibido

FE-2: Mesmo ortomapa selecionado para antes e depois

- **Condicao:** `antesId === depoisId`
- **Processamento funcional:** Backend processa normalmente, resultado sera zero mudancas
- **Efeito observavel na UI:** Mapa de mudancas vazio (sem areas detectadas)

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="mudancas"]').click()`
  - `page.locator('.tools-panel [value="mudancas"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="mudancas"] .form-group').nth(1).locator('.p-select').click()`
  - `page.locator('.p-select-option').last().click()`
  - `page.locator('.tools-panel [value="mudancas"] input[type="range"]').fill('20')`
  - `page.locator('.tools-panel [value="mudancas"] button').filter({ hasText: 'Detectar Mudancas' }).click()`
- **Asserts:**
  - `await expect(page.locator('.tools-panel [value="mudancas"] .form-group .p-select')).toHaveCount(2)`
- **Screenshots:**
  - `screenshot_uc009_01_aba_mudancas.png`
  - `screenshot_uc009_02_ortomapas_selecionados.png`
  - `screenshot_uc009_03_limiar_ajustado.png`
  - `screenshot_uc009_04_resultado_mapa.png`

UC-010: Agrupar Assinaturas Espectrais RGB com KMeans

Identificacao

- **ID:** UC-010
- **Modulo:** Analise
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Agrupar pixels de um ortomapa RGB por similaridade espectral com KMeans, definindo de 2 a 12 grupos e visualizando o raster sem atribuir classes semanticas nao comprovadas.

Pre-condicoes

- Projeto ativo com ortomapa RGB
- Sidebar direita visivel

Pos-condicoes

- Raster de grupos espectrais gerado e persistido
- Cada pixel atribuido a uma classe
- Camada classificada adicionada ao mapa com cores distintas por classe

Elementos de UI Envolvidos

- Tab "Cls" (classificacao) no ToolsPanel
- Select "Ortomapa"
- Select "Algoritmo" (K-Means RGB nao supervisionado)
- InputText "Numero de grupos espectrais"
- Checkbox e comandos da AOI

- Button "Gerar Grupos"

Fluxo Principal

Passo 1 - Navegar para aba Classificacao

- **Acao do ator:** Clica na aba "Cls"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `classificacao`, conteudo `<i class="pi pi-th-large"></i> Cls`
- **Resposta imediata do sistema:** Formulario de classificacao
- **Processamento funcional:** `activeTab = 'classificacao'`
- **Efeito observavel na UI:** Select de ortomapa, algoritmo, quantidade, AOI opcional e botao "Gerar Grupos"
- **Evidencia esperada:** Screenshot aba classificacao
- **Criterio de aceitacao:** Formulario visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="classificacao"]')`

Passo 2 - Selecionar ortomapa

- **Acao do ator:** Seleciona ortomapa RGB no dropdown
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select `classif.ortomapaId` com opcoes `rgbOptions`; DSM/DTM nao aparecem.
- **Resposta imediata do sistema:** Ortomapa selecionado
- **Processamento funcional:** `classif.ortomapaId` setado
- **Efeito observavel na UI:** Nome do ortomapa no dropdown
- **Evidencia esperada:** Screenshot
- **Criterio de aceitacao:** ID valido
- **Seletor Playwright:** `page.locator('.tools-panel [value="classificacao"] .form-group').first().locator('.p-select')`

Passo 3 - Verificar algoritmo KMeans

- **Acao do ator:** Verifica que "K-Means (Nao Supervisionado)" esta selecionado (padrao)
- **Contexto visual anterior:** Dropdown de algoritmo mostrando KMeans
- **Elemento de UI utilizado:** Select `classif.algorithm` com a opcao operacional `kmeans`.
- **Resposta imediata do sistema:** N/A - ja e padrao
- **Processamento funcional:** `classif.algorithm = 'kmeans'`
- **Efeito observavel na UI:** Dropdown mostra "K-Means RGB (Nao Supervisionado)". A interface nao anuncia classificacao supervisionada sem amostras reais.
- **Evidencia esperada:** Screenshot confirmando KMeans
- **Criterio de aceitacao:** Algoritmo KMeans selecionado, sem botao de treino
- **Seletor Playwright:** `page.locator('.tools-panel [value="classificacao"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Configurar numero de classes

- **Acao do ator:** Altera o numero de classes de 5 para 7
- **Contexto visual anterior:** Campo "Num. Classes" mostrando 5
- **Elemento de UI utilizado:** InputText com v-model.number `classif.numClasses`, type `number`
- **Resposta imediata do sistema:** Valor atualizado para 7
- **Processamento funcional:** `classif.numClasses = 7`
- **Efeito observavel na UI:** Campo mostra 7
- **Evidencia esperada:** Screenshot com 7 classes

- **Criterio de aceitacao:** Valor deve ser 7
- **Seletor Playwright:** `page.locator('.tools-panel [value="classificacao"] input[type="number"]')`

Passo 5 - Clicar em "Gerar Grupos"

- **Acao do ator:** Clica no botao "Gerar Grupos"
- **Contexto visual anterior:** Formulario completo
- **Elemento de UI utilizado:** Button "Gerar Grupos" com icon `pi pi-play`
- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** (1) Valida fonte RGB, projeto, quantidade e AOI. (2) POST `/api/tools/classify` com paths e IDs vinculados. (3) Treina KMeans deterministico em ate 100 mil pixels validos e prediz em blocos. (4) Gera GeoTIFF de rotulos 1..N, nodata=0. (5) Publica centros RGB, contagens, areas, percentuais, estilo e proveniencia.
- **Efeito observavel na UI:** Spinner no botao
- **Evidencia esperada:** Screenshot loading
- **Criterio de aceitacao:** Requisicao com parametros corretos
- **Seletor Playwright:** `page.locator('.tools-panel [value="classificacao"] button').filter({ hasText: 'Classificar' })`

Passo 6 - Visualizar mapa classificado

- **Acao do ator:** Observa resultado no mapa
- **Contexto visual anterior:** Mapa com ortomapa base
- **Elemento de UI utilizado:** MapViewer com camada de classificacao
- **Resposta imediata do sistema:** Mapa classificado com 7 cores distintas
- **Processamento funcional:** `addResultLayer('Classificacao', res.data)`. O estilo `getAnalysisStyle` do MapViewer atribui cor diferente por classe usando array `['#22c55e', '#3b82f6', '#f59e0b', '#ef4444', '#8b5cf6', '#ec4899']`
- **Efeito observavel na UI:** Mapa colorido por grupos espectrais, legenda com centro RGB e painel com percentual de cada grupo. Nenhum grupo recebe automaticamente o nome vegetacao, agua ou solo.
- **Evidencia esperada:** Screenshot da classificacao
- **Criterio de aceitacao:** Deve haver exatamente 7 classes de cores distintas visiveis no mapa
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Limitar a uma AOI

- **Passo divergente:** Antes da execucao
- **Acao:** Marca "Limitar a area" e desenha um poligono valido.
- **Efeito:** O botao fica bloqueado durante o desenho; o resultado e recortado e mascarado pela geometria, registrada na proveniencia.

Fluxos de Excecao

FE-1: Numero de classes invalido

- **Condicao:** Usuario seta classes para 0 ou 1
- **Processamento funcional:** Backend rejeita com erro 422
- **Efeito observavel na UI:** Erro logado, nenhuma camada adicionada

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="classificacao"]').click()`

    - `page.locator('.tools-panel [value="classificacao"] .form-group').first().locator('.p-select').click()`
    - `page.locator('.p-select-option').first().click()`
    - `page.locator('.tools-panel [value="classificacao"] input[type="number"]').fill('7')`
    - `page.locator('.tools-panel [value="classificacao"] button').filter({ hasText: 'Classificar' }).click()`
- **Asserts:**
    - `await expect(page.locator('.tools-panel [value="classificacao"] button').filter({ hasText: 'Desenhar Areas' })).toBeHidden()`
- **Screenshots:**
    - `screenshot_uc010_01_aba_classificacao.png`
    - `screenshot_uc010_02_kmeans_configurado.png`
    - `screenshot_uc010_03_resultado_mapa.png`

UC-011: Extrair Rede de Drenagem (Hidrologia)

Identificacao

- **ID:** UC-011
- **Modulo:** Analise
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Extrair canais potenciais a partir de um DTM projetado, com limiar de area contribuinte em m2, e visualizar a rede vetorial persistida sobre a ortofoto. A mesma aba permite delimitar uma bacia por exutorio e calcular TWI.

Posicionamento no produto: UC-011 e uma analise especializada adicional, executada depois que um produto de elevacao estiver disponivel. Nao recebe fotos brutas e nao faz parte do processamento fotogrametrico. O DTM pode ter sido produzido pelo ODM e importado no projeto (UC-025), ou pode vir de outra fonte, desde que esteja georreferenciado em CRS projetado com unidades metricas. O DSM e aceito com ressalva, pois vegetacao e construcoes podem alterar o escoamento modelado. Os resultados sao estimativas derivadas do relevo e exigem avaliacao adequada ao uso; a existencia de um DTM nao implica validacao hidrologica de campo.

Pre-condicoes

- Projeto ativo com DTM georreferenciado em CRS projetado; DSM permitido com aviso sobre sua adequacao
- Usuario autenticado com permissao de edicao no projeto

Pos-condicoes

- Rede potencial extraida como GeoJSON (LineStrings) WGS84
- Camada vetorial adicionada ao mapa
- Analise registrada

Elementos de UI Envolvidos

- Tab "Hid" (hidrologia) no ToolsPanel
- Select "Modelo de terreno (DTM recomendado)"
- Select "Analise" (Watershed, Streams, TWI)
- InputText "Area contribuinte minima (m2)"
- Button "Executar"

Fluxo Principal

Passo 1 - Navegar para aba Hidrologia

- **Acao do ator:** Clica na aba "Hid"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `hidrologia`, rotulo "Hid"
- **Resposta imediata do sistema:** Formulario de hidrologia
- **Processamento funcional:** `activeTab = 'hidrologia'`
- **Efeito observavel na UI:** Fonte, tipo de analise e botao "Executar"; campo de area apenas para drenagem
- **Evidencia esperada:** Screenshot aba hidrologia
- **Criterio de aceitacao:** Formulario visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="hidrologia"]')`

Passo 2 - Selecionar DTM

- **Acao do ator:** Seleciona DTM no dropdown
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select `hidro.ortomapaId` com opcoes `dsmDtmOptions`
- **Resposta imediata do sistema:** DTM selecionado
- **Processamento funcional:** `hidro.ortomapaId` setado
- **Efeito observavel na UI:** Nome do DTM no dropdown
- **Evidencia esperada:** Screenshot DTM selecionado
- **Criterio de aceitacao:** ID valido do projeto; DTM preferido ao DSM e aviso quando se escolhe DSM
- **Seletor Playwright:** `page.locator('.tools-panel [value="hidrologia"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar "Rede de Drenagem (Streams)"

- **Acao do ator:** Muda dropdown de analise para "Rede de Drenagem (Streams)"
- **Contexto visual anterior:** Dropdown mostrando "Bacias Hidrograficas (Watershed)" (padrao)
- **Elemento de UI utilizado:** Select `hidro.tipo`, seleciona `'streams'`
- **Resposta imediata do sistema:** Dropdown fecha. Campos de ponto de exutorio (latitude, longitude, botao Capturar do Mapa) NAO aparecem (so aparecem para watershed via `v-if="hidro.tipo === 'watershed'"`)
- **Processamento funcional:** `hidro.tipo = 'streams'`
- **Efeito observavel na UI:** Dropdown mostra "Rede de Drenagem (Streams)"
- **Evidencia esperada:** Screenshot com streams selecionado
- **Criterio de aceitacao:** Tipo deve ser 'streams'. Campos de pour point nao devem estar visiveis
- **Seletor Playwright:** `page.locator('.tools-panel [value="hidrologia"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Configurar area contribuinte

- **Acao do ator:** Mantem ou ajusta 50 m2 no campo de area contribuinte minima
- **Contexto visual anterior:** Campo "Area contribuinte minima (m2)" mostrando 50
- **Elemento de UI utilizado:** InputText com v-model.number `hidro.minimumArea`, type `number`
- **Resposta imediata do sistema:** Valor e preservado no formulario
- **Processamento funcional:** `hidro.minimumArea = 50`
- **Efeito observavel na UI:** Campo mostra 50
- **Evidencia esperada:** Screenshot com area configurada
- **Criterio de aceitacao:** Limiar positivo em m2; reduzir o valor aumenta a densidade de canais

- **Seletor Playwright:** `page.locator('.tools-panel [value="hidrologia"] input[type="number"]')`

Passo 5 - Clicar em "Executar"

- **Acao do ator:** Clica no botao "Executar"
- **Contexto visual anterior:** Formulario completo
- **Elemento de UI utilizado:** Button "Executar" com icon `pi pi-play`
- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** POST `/api/tools/hydrology/streams` com `{dtm_path, ortomapa_id, projeto_id, minimum_area_m2: 50, output_name}`. Backend prepara a grade, preenche depressoes por priority-flood, calcula D8 e acumulacao, converte area em m2 para celulas e vetoriza ligacoes como LineStrings.
- **Efeito observavel na UI:** Spinner no botao
- **Evidencia esperada:** Screenshot loading
- **Criterio de aceitacao:** HTTP 200, GeoJSON LineString, estatisticas e registro no catalogo
- **Seletor Playwright:** `page.locator('.tools-panel [value="hidrologia"] button').filter({ hasText: 'Executar' })`

Passo 6 - Verificar rede de drenagem no mapa

- **Acao do ator:** Observa o mapa
- **Contexto visual anterior:** Mapa com camadas anteriores
- **Elemento de UI utilizado:** MapViewer com camada GeoJSON
- **Resposta imediata do sistema:** Linhas de drenagem aparecem no mapa
- **Processamento funcional:** Resultado publicado em `analises`, adicionado ao mapa por `analysisVectorLayer`
- **Efeito observavel na UI:** Linhas azuis representando canais potenciais calculados sobre o DTM
- **Evidencia esperada:** Screenshot com rede de drenagem
- **Criterio de aceitacao:** Feicoes visiveis, contagem e comprimento exibidos; soma dos comprimentos reprojetados compativel com a estatistica publicada
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Usar Watershed em vez de Streams

- **Passo divergente:** Passo 3
- **Acao:** Seleciona "Bacias Hidrograficas (Watershed)"
- **Efeito:** Latitude/longitude WGS84 e raio de ajuste em metros aparecem. O usuario captura o ponto no mapa; o resultado e um raster binario persistido da bacia, com area, numero de celulas e distancia de ajuste do exutorio.

FA-2: Calcular TWI

- **Acao:** Seleciona "TWI - Indice Topografico" e executa com o DTM.
- **Efeito:** Raster colorizado persistido no mapa, com min, max, media e pixels validos. Indica propensao relativa a acumulacao, nao umidade medida.

Fluxos de Excecao

FE-1: Limiar muito alto (nenhum stream)

- **Condicao:** Area contribuinte maior que a acumulacao maxima do levantamento
- **Efeito:** HTTP 422 com erro visivel; nenhum produto vazio e publicado

FE-2: Ponto de exutorio invalido ou usuario externo

- **Condicao:** Latitude fora de -90 a 90, longitude fora de -180 a 180, ponto fora do DTM ou falta de permissao
- **Efeito:** HTTP 422 para ponto invalido ou HTTP 403 para usuario sem acesso; nenhum produto publicado

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="hidrologia"]').click()`
  - `page.locator('.tools-panel [value="hidrologia"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="hidrologia"] .form-group').nth(1).locator('.p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'Rede de Drenagem' }).click()`
  - `page.locator('.tools-panel input[type="number"]:visible').fill('50')`
  - `page.locator('.tools-panel [value="hidrologia"] button').filter({ hasText: 'Executar' }).click()`
- **Asserts:**
  - `await expect(page.locator('.tools-panel input[type="number"]:visible')).toHaveValue('50')`
- **Screenshots:**
  - `screenshot_uc011_01_aba_hidrologia.png`
  - `screenshot_uc011_02_streams_configurado.png`
  - `screenshot_uc011_03_resultado_mapa.png`

Validacao executada: `scripts/validar_hidrologia_fase5.py` com o DTM WebODM Aukerman, seis capturas e verificacoes independentes em `docs/validacao/hidrologia_fase5_2026-10-15.md`. O produto e uma estimativa hidrologica, sem calibracao de campo.

UC-012: Calcular Volume Acima/Abaixo de Referencia

Identificacao

- **ID:** UC-012
- **Modulo:** Analise
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Calcular o volume de material (corte e aterro) acima e abaixo de uma elevacao de referencia a partir de um DSM, obtendo valores em metros cubicos para planejamento de terraplanagem ou quantificacao de estoques.

Pre-condicoes

- Projeto ativo com ortomapa tipo DSM
- Sidebar direita visivel

Pos-condicoes

- Volumes de corte e aterro calculados e exibidos
- Volume liquido calculado
- Analise registrada

Elementos de UI Envolvidos

- Tab "Vol" (volume) no ToolsPanel
- Select "DSM"
- InputText "Elevacao de Referencia (m)"
- Button "Calcular Volume"
- Div `.result-stats` com Volume Corte, Volume Aterro, Volume Liquido

Fluxo Principal

Passo 1 - Navegar para aba Volume

- **Acao do ator:** Clica na aba "Vol"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `volume`, conteudo `<i class="pi pi-box"></i> Vol`
- **Resposta imediata do sistema:** Formulario de volume
- **Processamento funcional:** `activeTab = 'volume'`
- **Efeito observavel na UI:** Select de DSM, input de elevacao e botao
- **Evidencia esperada:** Screenshot aba volume
- **Criterio de aceitacao:** Formulario visivel
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="volume"]')`

Passo 2 - Selecionar DSM

- **Acao do ator:** Seleciona DSM no dropdown
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select `volume.ortomapaId` com opcoes `dsmDtmOptions`
- **Resposta imediata do sistema:** DSM selecionado
- **Processamento funcional:** `volume.ortomapaId` setado
- **Efeito observavel na UI:** Nome do DSM no dropdown
- **Evidencia esperada:** Screenshot DSM selecionado
- **Criterio de aceitacao:** ID valido
- **Seletor Playwright:** `page.locator('.tools-panel [value="volume"] .form-group').first().locator('.p-select')`

Passo 3 - Definir elevacao de referencia

- **Acao do ator:** Digita 850 no campo "Elevacao de Referencia (m)"
- **Contexto visual anterior:** Campo mostrando 0
- **Elemento de UI utilizado:** InputText com v-model.number `volume.refElevation`, type `number`
- **Resposta imediata do sistema:** Valor atualizado
- **Processamento funcional:** `volume.refElevation = 850`
- **Efeito observavel na UI:** Campo mostra 850
- **Evidencia esperada:** Screenshot com elevacao configurada
- **Criterio de aceitacao:** Valor deve ser 850
- **Seletor Playwright:** `page.locator('.tools-panel [value="volume"] input[type="number"]')`

Passo 4 - Clicar em "Calcular Volume"

- **Acao do ator:** Clica no botao
- **Contexto visual anterior:** DSM e elevacao configurados
- **Elemento de UI utilizado:** Button "Calcular Volume" com icon `pi pi-play`

- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** POST `/api/tools/volume` com `{ortomapa_id, ref_elevation: 850}`. Backend: (1) Le raster DSM. (2) Para cada pixel, calcula diferenca entre elevacao e referencia. (3) Pixels acima: volume de corte = soma(diff * area_pixel). (4) Pixels abaixo: volume de aterro = soma(|diff| * area_pixel). (5) Volume liquido = corte - aterro. (6) Retorna `{cut, fill, net}`
- **Efeito observavel na UI:** Spinner
- **Evidencia esperada:** Screenshot loading
- **Criterio de aceitacao:** Requisicao correta
- **Seletor Playwright:** `page.locator('.tools-panel [value="volume"] button').filter({ hasText: 'Calcular Volume' })`

Passo 5 - Verificar resultados de volume

- **Acao do ator:** Observa os resultados
- **Contexto visual anterior:** Loading
- **Elemento de UI utilizado:** Div `.result-stats` com tres `.stat-row`: Volume Corte (m3), Volume Aterro (m3), Volume Liquido (m3)
- **Resposta imediata do sistema:** Valores numericos formatados com 1 casa decimal aparecem
- **Processamento funcional:** `volume.result = res.data`. Template exibe `{{ volume.result.cut?.toFixed(1) }} m3`, `{{ volume.result.fill?.toFixed(1) }} m3`, `{{ volume.result.net?.toFixed(1) }} m3`
- **Efeito observavel na UI:** Bloco verde claro com: "Volume Corte: XXXX.X m3", "Volume Aterro: YYYY.Y m3", "Volume Liquido: ZZZZ.Z m3"
- **Evidencia esperada:** Screenshot com volumes
- **Criterio de aceitacao:** Tres valores numericos formatados. Net = Cut - Fill (com sinal)
- **Seletor Playwright:** `page.locator('.tools-panel .result-stats')`

Fluxos Alternativos

FA-1: Elevacao de referencia zero

- **Passo divergente:** Passo 3
- **Acao:** Manter 0 como referencia
- **Efeito:** Todo o volume do DSM sera "corte" (acima de zero)

Fluxos de Excecao

FE-1: DSM nao selecionado

- **Condicao:** `volume.ortomapaId` e null
- **Processamento funcional:** Retorno silencioso

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="volume"]').click()`
  - `page.locator('.tools-panel [value="volume"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="volume"] input[type="number"]').fill('850')`
  - `page.locator('.tools-panel [value="volume"] button').filter({ hasText: 'Calcular Volume' }).click()`
  - `page.waitForSelector('.tools-panel .result-stats')`
- **Asserts:**
  - `await expect(page.locator('.result-stats .stat-row')).toHaveCount(3)`

    - `await expect(page.locator('.result-stats')).toContainText('Volume Corte')`
    - `await expect(page.locator('.result-stats')).toContainText('Volume Aterro')`
    - `await expect(page.locator('.result-stats')).toContainText('Volume Liquido')`
- **Screenshots:**
    - `screenshot_uc012_01_aba_volume.png`
    - `screenshot_uc012_02_configurado.png`
    - `screenshot_uc012_03_resultados.png`

UC-013: Criar Anotacao Poligono no Mapa

Identificacao

- **ID:** UC-013
- **Modulo:** Anotacoes
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Desenhar um poligono no mapa para criar uma anotacao espacial vinculada a um ortomapa, definindo categoria, rotulo e descricao, para documentar areas de interesse como vegetacao, erosao ou construcoes.

Pre-condicoes

- Projeto ativo com pelo menos um ortomapa
- Sidebar direita visivel com DrawTools
- Mapa interativo funcional

Pos-condicoes

- Anotacao salva no banco com geometria WKT
- Poligono visivel no mapa como feature GeoJSON
- Anotacao listada na secao "Anotacoes Existentes" do DrawTools

Elementos de UI Envolvidos

- Secao "Anotacoes" (DrawTools) na sidebar direita
- Botoes de desenho: Ponto (pi-map-marker), Linha (pi-minus), Poligono (pi-stop), Retangulo (pi-clone), Cancelar (pi-times)
- Formulario de anotacao (`.annotation-form`): Select categoria, InputText rotulo, Textarea descricao
- Botoes "Cancelar" e "Salvar" no formulario
- Lista `.annotations-list` com `.annotation-item`
- MapViewer com GeoJSON de anotacoes

Fluxo Principal

Passo 1 - Ativar modo de desenho Poligono

- **Acao do ator:** Clica no botao de poligono (icone quadrado pi-stop) na secao Anotacoes
- **Contexto visual anterior:** Sidebar direita mostrando secao "Anotacoes" com 5 botoes de desenho, todos em estado secundario (cinza)

- **Elemento de UI utilizado:** Button PrimeVue com icon `pi pi-stop`, severity condicional (success se ativo, secondary se nao), tamanho `small`, v-tooltip "Poligono"
- **Resposta imediata do sistema:** O botao de poligono muda para severity `success` (verde), indicando modo ativo
- **Processamento funcional:** `setMode('polygon')` chama `mapStore.setDrawMode('polygon')`. Se ja estava em modo polygon, desativa (toggle). `setDrawMode` tambem desativa `measureMode` se estiver ativo
- **Efeito observavel na UI:** Botao poligono fica verde. Cursor do mapa muda para crosshair. Botao "Cancelar" (pi-times) fica habilitado
- **Evidencia esperada:** Screenshot mostrando botao poligono ativo (verde)
- **Criterio de aceitacao:** `mapStore.drawMode` deve ser `'polygon'`. Botao poligono deve ter severity `success`
- **Seletor Playwright:** `page.locator('.draw-tools .draw-buttons button').nth(2)`

Passo 2 - Desenhar poligono no mapa (cliques nos vertices)

- **Acao do ator:** Clica em 4+ pontos no mapa para definir os vertices do poligono
- **Contexto visual anterior:** Mapa com cursor em crosshair, modo de desenho ativo
- **Elemento de UI utilizado:** `l-map` no MapViewer, evento `@click` que chama `onMapClick`
- **Resposta imediata do sistema:** Cada clique adiciona um vertice visivel no mapa. Linhas conectam os vertices
- **Processamento funcional:** O MapViewer captura os cliques e acumula coordenadas. O sistema de desenho (Leaflet.draw ou implementacao custom) renderiza o poligono em construcao
- **Efeito observavel na UI:** Vertices marcados com circulos, lados do poligono desenhados como linhas, area sombreada
- **Evidencia esperada:** Screenshot do poligono em construcao
- **Criterio de aceitacao:** Pelo menos 3 vertices devem estar visiveis. Poligono deve ser desenhado
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 300, y: 200 } })`

Passo 3 - Finalizar o desenho (duplo-clique ou fechar poligono)

- **Acao do ator:** Duplo-clica ou clica no primeiro vertice para fechar o poligono
- **Contexto visual anterior:** Poligono com 4+ vertices em construcao
- **Elemento de UI utilizado:** Mapa (evento de finalizacao de desenho)
- **Resposta imediata do sistema:** O poligono e finalizado e o formulario de anotacao aparece na sidebar
- **Processamento funcional:** `onDrawComplete(geometry)` e chamado com a geometria WKT. `drawnGeometry` e setado. `showForm = true`. `mapStore.setDrawMode(null)` desativa o modo de desenho
- **Efeito observavel na UI:** Formulario "Nova Anotacao" aparece na sidebar com campos categoria, rotulo e descricao. O botao de poligono volta a severity secondary
- **Evidencia esperada:** Screenshot do formulario de anotacao aberto
- **Criterio de aceitacao:** Formulario `.annotation-form` deve estar visivel com titulo "Nova Anotacao"
- **Seletor Playwright:** `page.locator('.draw-tools .annotation-form')`

Passo 4 - Preencher dados da anotacao

- **Acao do ator:** Seleciona categoria "Erosao", digita rotulo "Area de erosao ativa" e descricao "Vocorocas identificadas em marco 2026"
- **Contexto visual anterior:** Formulario com campos vazios (categoria padrao: vegetacao)
- **Elemento de UI utilizado:** Select de categoria (opcoes: Vegetacao, Construcao, Agua, Solo Exposto, Erosao, Infraestrutura). InputText de rotulo (placeholder "Nome da anotacao"). Textarea de descricao (2 linhas)
- **Resposta imediata do sistema:** Campos preenchidos conforme digitacao
- **Processamento funcional:** `form.categoria = 'erosao'`, `form.rotulo = 'Area de erosao ativa'`, `form.descricao = 'Vocorocas identificadas em marco 2026'`
- **Efeito observavel na UI:** Formulario completo com dados
- **Evidencia esperada:** Screenshot do formulario preenchido
- **Criterio de aceitacao:** Todos os campos devem conter os valores informados

- **Seletor Playwright:** `page.locator('.annotation-form .form-group').first().locator('.p-select')` (categoria)

Passo 5 - Salvar a anotacao

- **Acao do ator:** Clica no botao "Salvar"
- **Contexto visual anterior:** Formulario preenchido
- **Elemento de UI utilizado:** Button "Salvar" com icon `pi pi-check`, tamanho `small`, dentro de `.form-actions`
- **Resposta imediata do sistema:** Formulario fecha
- **Processamento funcional:** (1) Verifica `drawnGeometry` e `projectStore.activeProject`. (2) Pega primeiro ortomapa disponivel. (3) POST `/api/anotacoes` com `{ortomapa_id, categoria: 'erosao', rotulo: 'Area de erosao ativa', descricao: '...', geometria_wkt: 'POLYGON(...)'}`. (4) `projectStore.fetchAnotacoes()` recarrega lista. (5) Reseta formulario
- **Efeito observavel na UI:** (1) Formulario desaparece. (2) Nova entrada na lista "Anotacoes Existentes". (3) Poligono permanece no mapa com cor da categoria erosao (#f97316, laranja)
- **Evidencia esperada:** Screenshot mostrando anotacao na lista e no mapa
- **Criterio de aceitacao:** Anotacao deve aparecer em `.annotations-list` com rotulo "Area de erosao ativa" e categoria "erosao". Cor do indicador deve ser laranja
- **Seletor Playwright:** `page.locator('.annotation-form .form-actions button').filter({ hasText: 'Salvar' })`

Passo 6 - Verificar anotacao no mapa

- **Acao do ator:** Clica no poligono de anotacao no mapa
- **Contexto visual anterior:** Poligono laranja visivel no mapa
- **Elemento de UI utilizado:** `l-geo-json` de anotacoes no MapViewer com `onEachAnotacao` que cria popup
- **Resposta imediata do sistema:** Popup Leaflet aparece com rotulo, categoria e descricao
- **Processamento funcional:** `onEachAnotacao(feature, layer)` chama `layer.bindPopup()` com HTML contendo rotulo, categoria e descricao
- **Efeito observavel na UI:** Popup com: "**Area de erosao ativa**", "*erosao*", "Vocorocas identificadas em marco 2026"
- **Evidencia esperada:** Screenshot do popup aberto
- **Criterio de aceitacao:** Popup deve exibir as informacoes corretas da anotacao
- **Seletor Playwright:** `page.locator('.leaflet-popup-content')`

Fluxos Alternativos

FA-1: Cancelar a anotacao antes de salvar

- **Passo divergente:** Passo 5
- **Acao:** Clica em "Cancelar" em vez de "Salvar"
- **Processamento funcional:** `cancelAnnotation()` reseta formulario, `showForm = false`, `drawnGeometry = null`
- **Efeito observavel na UI:** Formulario fecha, poligono desenhado desaparece, nenhuma anotacao salva

Fluxos de Excecao

FE-1: Nenhum ortomapa disponivel

- **Condicao:** `projectStore.ortomapas` esta vazio
- **Processamento funcional:** `saveAnnotation` detecta que nao ha ortomapa: `if (!ortomapa)` loga erro e retorna
- **Efeito observavel na UI:** A anotacao nao e salva. Erro no console

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.draw-tools .draw-buttons button').nth(2).click()`
  - `page.locator('.leaflet-container').click({ position: { x: 300, y: 200 } })`

    - `page.locator('.leaflet-container').click({ position: { x: 400, y: 200 } })`
    - `page.locator('.leaflet-container').click({ position: { x: 400, y: 300 } })`
    - `page.locator('.leaflet-container').dblclick({ position: { x: 300, y: 300 } })`
    - `page.locator('.annotation-form .p-select').click()`
    - `page.locator('.p-select-option').filter({ hasText: 'Erosao' }).click()`
    - `page.locator('.annotation-form input').fill('Area de erosao ativa')`
    - `page.locator('.annotation-form textarea').fill('Vocorocas identificadas em marco 2026')`
    - `page.locator('.annotation-form .form-actions button').filter({ hasText: 'Salvar' }).click()`
- **Asserts:**
    - `await expect(page.locator('.annotation-form')).toBeVisible()` (apos passo 3)
    - `await expect(page.locator('.annotation-item .annotation-label')).toContainText('Area de erosao ativa')`
- **Screenshots:**
    - `screenshot_uc013_01_modo_poligono_ativo.png`
    - `screenshot_uc013_02_poligono_desenhado.png`
    - `screenshot_uc013_03_formulario_preenchido.png`
    - `screenshot_uc013_04_anotacao_salva.png`
    - `screenshot_uc013_05_popup_mapa.png`

UC-014: Criar Anotacao Ponto no Mapa

Identificacao

- **ID:** UC-014
- **Modulo:** Anotacoes
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Marcar um ponto de interesse no mapa como anotacao, definindo categoria, rotulo e descricao, para documentar localizacoes especificas como pontos de coleta, marcos ou ocorrencias.

Pre-condicoes

- Projeto ativo com pelo menos um ortomapa
- Sidebar direita visivel com DrawTools

Pos-condicoes

- Anotacao de ponto salva com geometria POINT
- Marcador visivel no mapa
- Anotacao na lista

Elementos de UI Envolvidos

- Botao de ponto (pi-map-marker) no DrawTools
- Formulario de anotacao
- MapViewer com marker

Fluxo Principal

Passo 1 - Ativar modo de desenho Ponto

- **Acao do ator:** Clica no botao de ponto (icone pi-map-marker)
- **Contexto visual anterior:** Botoes de desenho em estado secundario
- **Elemento de UI utilizado:** Button com icon `pi pi-map-marker`, primeiro botao em `.draw-buttons`
- **Resposta imediata do sistema:** Botao fica verde (severity success)
- **Processamento funcional:** `setMode('point')` chama `mapStore.setDrawMode('point')`
- **Efeito observavel na UI:** Botao de ponto verde, cursor crosshair no mapa
- **Evidencia esperada:** Screenshot botao ativo
- **Criterio de aceitacao:** `mapStore.drawMode === 'point'`
- **Seletor Playwright:** `page.locator('.draw-tools .draw-buttons button').first()`

Passo 2 - Clicar no mapa para posicionar o ponto

- **Acao do ator:** Clica em uma posicao especifica no mapa
- **Contexto visual anterior:** Cursor crosshair sobre o mapa
- **Elemento de UI utilizado:** `l-map` com evento de clique
- **Resposta imediata do sistema:** Um marcador aparece na posicao clicada
- **Processamento funcional:** O evento de clique captura lat/lng e cria geometria POINT(lng lat)
- **Efeito observavel na UI:** Marcador no mapa, formulario de anotacao abre na sidebar
- **Evidencia esperada:** Screenshot com marcador e formulario
- **Criterio de aceitacao:** Formulario `.annotation-form` visivel com titulo "Nova Anotacao"
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 350, y: 250 } })`

Passo 3 - Selecionar categoria "Infraestrutura"

- **Acao do ator:** Seleciona "Infraestrutura" no dropdown de categoria
- **Contexto visual anterior:** Formulario com categoria padrao "Vegetacao"
- **Elemento de UI utilizado:** Select de categorias com opcoes definidas
- **Resposta imediata do sistema:** Categoria selecionada
- **Processamento funcional:** `form.categoria = 'infraestrutura'`
- **Efeito observavel na UI:** Dropdown mostra "Infraestrutura"
- **Evidencia esperada:** Screenshot com categoria
- **Criterio de aceitacao:** Categoria deve ser 'infraestrutura'
- **Seletor Playwright:** `page.locator('.annotation-form .p-select')`

Passo 4 - Preencher rotulo e descricao

- **Acao do ator:** Digita "Marco geodesico P01" no rotulo e "Ponto de controle para georreferenciamento" na descricao
- **Contexto visual anterior:** Campos de rotulo e descricao vazios
- **Elemento de UI utilizado:** InputText de rotulo e Textarea de descricao
- **Resposta imediata do sistema:** Campos preenchidos
- **Processamento funcional:** `form.rotulo` e `form.descricao` atualizados
- **Efeito observavel na UI:** Dados nos campos
- **Evidencia esperada:** Screenshot formulario completo
- **Criterio de aceitacao:** Campos devem conter os valores digitados
- **Seletor Playwright:** `page.locator('.annotation-form input').fill('Marco geodesico P01')`

Passo 5 - Salvar anotacao

- **Acao do ator:** Clica em "Salvar"
- **Contexto visual anterior:** Formulario completo
- **Elemento de UI utilizado:** Button "Salvar"
- **Resposta imediata do sistema:** Formulario fecha
- **Processamento funcional:** POST `/api/anotacoes` com geometria POINT. Recarrega anotacoes
- **Efeito observavel na UI:** Formulario fecha, anotacao aparece na lista com cor roxa (infraestrutura: #8b5cf6) e no mapa como ponto roxo
- **Evidencia esperada:** Screenshot com anotacao salva
- **Criterio de aceitacao:** Anotacao com rotulo "Marco geodesico P01" na lista, cor roxa
- **Seletor Playwright:** `page.locator('.annotation-form .form-actions button').filter({ hasText: 'Salvar' })`

Fluxos Alternativos

FA-1: Cancelar antes de posicionar ponto

- **Acao:** Clica no botao Cancelar (pi-times) antes de clicar no mapa
- **Processamento:** `cancelDraw()` desativa modo, reseta formulario
- **Efeito:** Modo de desenho desativado, nada salvo

Fluxos de Excecao

FE-1: Erro de rede ao salvar

- **Condicao:** Backend indisponivel
- **Efeito:** Erro logado, anotacao nao salva, formulario pode permanecer aberto

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.draw-tools .draw-buttons button').first().click()`
  - `page.locator('.leaflet-container').click({ position: { x: 350, y: 250 } })`
  - `page.locator('.annotation-form .p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'Infraestrutura' }).click()`
  - `page.locator('.annotation-form input').fill('Marco geodesico P01')`
  - `page.locator('.annotation-form textarea').fill('Ponto de controle para georreferenciamento')`
  - `page.locator('.annotation-form .form-actions button').filter({ hasText: 'Salvar' }).click()`
- **Asserts:**
  - `await expect(page.locator('.annotation-item')).toHaveCount(previousCount + 1)`
  - `await expect(page.locator('.annotation-item .annotation-label').last()).toContainText('Marco geodesico P01')`
- **Screenshots:**
  - `screenshot_uc014_01_modo_ponto.png`
  - `screenshot_uc014_02_ponto_no_mapa.png`
  - `screenshot_uc014_03_formulario.png`
  - `screenshot_uc014_04_anotacao_salva.png`

UC-015: Medir Distancia no Mapa

Identificacao

- **ID:** UC-015

- **Modulo:** Mapa
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Medir a distancia entre dois ou mais pontos no mapa utilizando a ferramenta de medicao, obtendo distancia total e distancias parciais por segmento em metros ou quilometros.

Pre-condicoes

- Mapa interativo funcional
- MeasureTools disponivel (ativado via mapStore.measureMode)

Pos-condicoes

- Distancia total e segmentos exibidos no painel de medicao
- Linha de medicao visivel no mapa
- Pontos marcados no mapa

Elementos de UI Envolvidos

- Painel de medicao `.measure-panel` (componente MeasureTools)
- Botoes de modo: "Distancia" e "Area" (`.mode-btn`)
- Resultado `.measure-result` com `.result-value` (distancia total)
- Detalhe por segmento
- Botoes "Desfazer" e "Limpar" (`.action-btn`)
- Dica de uso `.measure-hint`
- Polyline no MapViewer (laranja, tracejado)
- CircleMarkers nos pontos de medicao

Fluxo Principal

Passo 1 - Ativar modo de medicao de distancia

- **Acao do ator:** Ativa o modo de medicao (via interface - mapStore.setMeasureMode('distance'))
- **Contexto visual anterior:** Mapa normal sem painel de medicao
- **Elemento de UI utilizado:** O painel MeasureTools aparece quando `mapStore.measureMode` e setado
- **Resposta imediata do sistema:** Painel `.measure-panel` aparece no canto superior esquerdo do mapa com botoes "Distancia" (ativo) e "Area"
- **Processamento funcional:** `mapStore.setMeasureMode('distance')` seta `measureMode = 'distance'` e desativa `drawMode` . O MeasureTools renderiza condicionalmente (`v-if="mapStore.measureMode"`)
- **Efeito observavel na UI:** Painel de medicao com titulo "Medir Distancia", botao "Distancia" ativo (azul), resultado "0 m", dica "Clique no mapa para adicionar pontos"
- **Evidencia esperada:** Screenshot do painel de medicao
- **Criterio de aceitacao:** Painel visivel, modo distancia ativo
- **Seletor Playwright:** `page.locator('.measure-panel .mode-btn').first()`

Passo 2 - Clicar no primeiro ponto

- **Acao do ator:** Clica em um ponto no mapa
- **Contexto visual anterior:** Mapa limpo, painel mostrando "0 m"
- **Elemento de UI utilizado:** `l-map` evento `@click` no MapViewer

- **Resposta imediata do sistema:** Um circulo amarelo aparece no ponto clicado
- **Processamento funcional:** `onMapClick` detecta `mapStore.measureMode` e adiciona `[lat, lng]` a `measurePoints`. Um `l-circle-marker` e renderizado
- **Efeito observavel na UI:** Circulo amarelo no ponto, "1 ponto(s) marcado(s)" no detalhe, distancia ainda "0 m"
- **Evidencia esperada:** Screenshot com primeiro ponto
- **Criterio de aceitacao:** Um ponto marcado, distancia 0
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 200, y: 200 } })`

Passo 3 - Clicar no segundo ponto

- **Acao do ator:** Clica em outro ponto no mapa
- **Contexto visual anterior:** Um ponto marcado
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Segundo circulo aparece, linha tracejada laranja conecta os dois pontos
- **Processamento funcional:** Segundo ponto adicionado a `measurePoints`. `l-polyline` renderiza com `:latLngs="measurePoints"`, cor `#f59e0b`, peso 3, tracejado `8,6`. Computed `segments` calcula distancia Haversine entre os pontos. `totalDistance` e a soma dos segmentos
- **Efeito observavel na UI:** Linha tracejada laranja entre os pontos. Distancia total exibida (ex: "156.32 m"). "Segmento 1: 156.32 m" nos detalhes
- **Evidencia esperada:** Screenshot com linha e distancia
- **Criterio de aceitacao:** Distancia deve ser > 0 e formatada em metros ou km
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 400, y: 300 } })`

Passo 4 - Adicionar terceiro ponto

- **Acao do ator:** Clica em mais um ponto
- **Contexto visual anterior:** Dois pontos com linha
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Terceiro circulo, segunda linha tracejada
- **Processamento funcional:** Terceiro ponto adicionado. Polyline agora com 3 pontos. Dois segmentos calculados. Distancia total = soma dos dois
- **Efeito observavel na UI:** Polilinha com 3 pontos. "Segmento 1: X m", "Segmento 2: Y m". Distancia total = X + Y
- **Evidencia esperada:** Screenshot com 3 pontos
- **Criterio de aceitacao:** Distancia total deve ser soma dos segmentos. "3 ponto(s) marcado(s)"
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 500, y: 200 } })`

Passo 5 - Desfazer ultimo ponto

- **Acao do ator:** Clica no botao "Desfazer"
- **Contexto visual anterior:** 3 pontos com 2 segmentos
- **Elemento de UI utilizado:** Button `.action-btn.undo` com icone `pi pi-undo` e texto "Desfazer"
- **Resposta imediata do sistema:** Terceiro ponto removido, polilinha volta a ter 2 pontos
- **Processamento funcional:** Emite evento `undo-point`. O pai (App/MapViewer) remove ultimo ponto do array `measurePoints`
- **Efeito observavel na UI:** Terceiro circulo desaparece, segunda linha desaparece, distancia total volta a ser apenas segmento 1
- **Evidencia esperada:** Screenshot com 2 pontos
- **Criterio de aceitacao:** Deve ter voltado para 2 pontos e 1 segmento
- **Seletor Playwright:** `page.locator('.measure-panel .action-btn.undo')`

Passo 6 - Fechar medicao

- **Acao do ator:** Clica no botao X (fechar) do painel
- **Contexto visual anterior:** Painel de medicao aberto
- **Elemento de UI utilizado:** Button `.close-btn` no `.measure-header`
- **Resposta imediata do sistema:** Painel desaparece, pontos e linhas de medicao sao limpos
- **Processamento funcional:** `closeMeasure()` chama `mapStore.setMeasureMode(null)` e emite `clear-points`
- **Efeito observavel na UI:** Painel desaparece, mapa volta ao normal sem linhas de medicao
- **Evidencia esperada:** Screenshot mapa limpo
- **Criterio de aceitacao:** `mapStore.measureMode` deve ser null. Painel nao visivel
- **Seletor Playwright:** `page.locator('.measure-panel .close-btn')`

Fluxos Alternativos

FA-1: Limpar todos os pontos

- **Passo divergente:** Passo 5
- **Acao:** Clica em "Limpar" em vez de "Desfazer"
- **Processamento:** `clearMeasurements()` emite `clear-points`
- **Efeito:** Todos os pontos removidos, distancia volta a "0 m"

Fluxos de Excecao

FE-1: Apenas um ponto marcado

- **Condicao:** Apenas 1 ponto no mapa
- **Efeito:** Distancia permanece "0 m", nenhum segmento listado

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.measure-panel .mode-btn').first().click()` (se nao esta ativo)
  - `page.locator('.leaflet-container').click({ position: { x: 200, y: 200 } })`
  - `page.locator('.leaflet-container').click({ position: { x: 400, y: 300 } })`
  - `page.locator('.leaflet-container').click({ position: { x: 500, y: 200 } })`
  - `page.locator('.measure-panel .action-btn.undo').click()`
  - `page.locator('.measure-panel .close-btn').click()`
- **Asserts:**
  - `await expect(page.locator('.measure-panel .result-value')).not.toContainText('0 m')` (apos 2 pontos)
  - `await expect(page.locator('.measure-panel .segment')).toHaveCount(2)` (apos 3 pontos)
  - `await expect(page.locator('.measure-panel')).toBeHidden()` (apos fechar)
- **Screenshots:**
  - `screenshot_uc015_01_painel_medicao.png`
  - `screenshot_uc015_02_primeiro_ponto.png`
  - `screenshot_uc015_03_distancia_dois_pontos.png`
  - `screenshot_uc015_04_tres_pontos.png`
  - `screenshot_uc015_05_apos_desfazer.png`

UC-016: Medir Area no Mapa

Identificacao

- **ID:** UC-016
- **Modulo:** Mapa
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Medir a area de uma regiao poligonal no mapa clicando nos vertices, obtendo area em metros quadrados ou hectares e perimetro em metros ou quilometros.

Pre-condicoes

- Mapa interativo funcional
- MeasureTools disponivel

Pos-condicoes

- Area e perimetro exibidos no painel
- Poligono semi-transparente visivel no mapa

Elementos de UI Envolvidos

- MeasureTools com modo "Area"
- Botao `.mode-btn` "Area"
- `.measure-result` com area e perimetro
- `l-polygon` no MapViewer (laranja, semi-transparente)

Fluxo Principal

Passo 1 - Ativar modo de medicao de area

- **Acao do ator:** Clica no botao "Area" no painel de medicao
- **Contexto visual anterior:** Painel de medicao pode estar em modo distancia ou acabou de ser ativado
- **Elemento de UI utilizado:** Button `.mode-btn` com icone `pi pi-stop` e texto "Area"
- **Resposta imediata do sistema:** Botao "Area" fica ativo (azul), botao "Distancia" fica inativo
- **Processamento funcional:** `mapStore.setMeasureMode('area')`. Se estava em distancia, pontos sao limpos (watch no MeasureTools emite `clear-points` quando modo muda)
- **Efeito observavel na UI:** Botao Area com fundo azul (#89b4fa), label "Area" no resultado, dica "Minimo 3 pontos para calcular area"
- **Evidencia esperada:** Screenshot modo area ativo
- **Criterio de aceitacao:** `mapStore.measureMode === 'area'`. Botao Area com classe `.active`
- **Seletor Playwright:** `page.locator('.measure-panel .mode-btn').last()`

Passo 2 - Marcar primeiro vertice

- **Acao do ator:** Clica no mapa
- **Contexto visual anterior:** Mapa limpo
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Circulo amarelo no ponto

- **Processamento funcional:** Ponto adicionado a `measurePoints`
- **Efeito observavel na UI:** 1 vertice marcado, area "0 m2"
- **Evidencia esperada:** Screenshot com primeiro vertice
- **Criterio de aceitacao:** 1 vertice, area 0
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 200, y: 200 } })`

Passo 3 - Marcar segundo vertice

- **Acao do ator:** Clica em segundo ponto
- **Contexto visual anterior:** 1 vertice marcado
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Segundo circulo
- **Processamento funcional:** Segundo ponto adicionado. Area ainda 0 (precisa de 3 pontos minimo)
- **Efeito observavel na UI:** 2 vertices, area "0 m2"
- **Evidencia esperada:** Screenshot com 2 vertices
- **Criterio de aceitacao:** 2 vertices marcados
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 400, y: 200 } })`

Passo 4 - Marcar terceiro vertice (forma poligono)

- **Acao do ator:** Clica em terceiro ponto
- **Contexto visual anterior:** 2 vertices
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Poligono semi-transparente aparece, area calculada
- **Processamento funcional:** Terceiro ponto adicionado. `l-polygon` renderiza com `:latLngs="measurePoints"`, cor `#f59e0b`, fillOpacity 0.2. `formattedArea` calcula via `polygonArea()` usando projecao plana local e formula de Shoelace. `formattedPerimeter` soma distancias Haversine de todos os lados incluindo fechamento
- **Efeito observavel na UI:** Poligono triangular laranja semi-transparente. Area exibida (ex: "1234.56 m2" ou "0.1234 ha" se > 10000). Perimetro exibido (ex: "234.56 m")
- **Evidencia esperada:** Screenshot com poligono e area
- **Criterio de aceitacao:** Area > 0, perimetro > 0, poligono visivel
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 300, y: 400 } })`

Passo 5 - Adicionar quarto vertice (refinar poligono)

- **Acao do ator:** Clica em quarto ponto
- **Contexto visual anterior:** Triangulo
- **Elemento de UI utilizado:** `l-map` clique
- **Resposta imediata do sistema:** Poligono se expande para quadrilatero
- **Processamento funcional:** Quarto ponto. Poligono redesenhado. Area e perimetro recalculados
- **Efeito observavel na UI:** Quadrilatero com area maior e perimetro atualizado. "4 vertice(s)"
- **Evidencia esperada:** Screenshot com quadrilatero
- **Criterio de aceitacao:** Area deve ser diferente (maior) que com 3 vertices
- **Seletor Playwright:** `page.locator('.leaflet-container').click({ position: { x: 100, y: 400 } })`

Passo 6 - Limpar medicao

- **Acao do ator:** Clica em "Limpar"
- **Contexto visual anterior:** Quadrilatero com area
- **Elemento de UI utilizado:** Button `.action-btn.clear` com icone `pi pi-eraser` e texto "Limpar"
- **Resposta imediata do sistema:** Poligono e pontos removidos
- **Processamento funcional:** `clearMeasurements()` emite `clear-points`. Array `measurePoints` e esvaziado

- **Efeito observavel na UI:** Mapa limpo, area volta a "0 m2", perimetro "0 m"
- **Evidencia esperada:** Screenshot limpo
- **Criterio de aceitacao:** Nenhum ponto ou poligono no mapa, area "0 m2"
- **Seletor Playwright:** `page.locator('.measure-panel .action-btn.clear')`

Fluxos Alternativos

FA-1: Area grande (em hectares)

- **Condicao:** Area medida > 10000 m2
- **Efeito:** Exibicao automaticamente muda para hectares com 4 casas decimais (ex: "1.2345 ha")

Fluxos de Excecao

FE-1: Menos de 3 pontos

- **Condicao:** Apenas 1 ou 2 pontos marcados
- **Efeito:** Area mostra "0 m2", nenhum poligono renderizado (l-polygon requer 3+ pontos)

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.measure-panel .mode-btn').last().click()`
  - `page.locator('.leaflet-container').click({ position: { x: 200, y: 200 } })`
  - `page.locator('.leaflet-container').click({ position: { x: 400, y: 200 } })`
  - `page.locator('.leaflet-container').click({ position: { x: 300, y: 400 } })`
  - `page.locator('.leaflet-container').click({ position: { x: 100, y: 400 } })`
  - `page.locator('.measure-panel .action-btn.clear').click()`
- **Asserts:**
  - `await expect(page.locator('.measure-result .result-value')).not.toContainText('0')` (apos 3 pontos)
  - `await expect(page.locator('.measure-result')).toContainText('Perimetro')`
- **Screenshots:**
  - `screenshot_uc016_01_modo_area.png`
  - `screenshot_uc016_02_triangulo.png`
  - `screenshot_uc016_03_quadrilatero.png`
  - `screenshot_uc016_04_limpo.png`

UC-017: Comparar Dois Ortomapas (Swipe)

Identificacao

- **ID:** UC-017
- **Modulo:** Mapa
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Comparar visualmente dois ortomapas de datas diferentes usando a ferramenta de swipe (deslizar), permitindo ao usuario arrastar uma barra vertical para revelar um ortomapa ou outro em cada lado.

Pre-condicoes

- Projeto ativo com pelo menos dois ortomapas
- mapStore.compareMode ativado

Pos-condicoes

- Dois ortomapas exibidos no modo de comparacao
- Slider funcional para revelar cada lado

Elementos de UI Envolvidos

- CompareView substituindo MapViewer (condicional `v-if="mapStore.compareMode"`)
- Select "Antes" e Select "Depois" na barra de controles
- Botao toggle modo (Deslizar / Lado a Lado)
- Botao "Fechar"
- Slider vertical arrastavel (`.swipe-slider`)
- Labels "Antes" e "Depois"

Fluxo Principal

Passo 1 - Ativar modo de comparacao

- **Acao do ator:** Ativa o modo de comparacao (via `mapStore.setCompareMode(true)`)
- **Contexto visual anterior:** MapViewer normal exibido
- **Elemento de UI utilizado:** A ativacao pode ser via botao na toolbar ou API direta do store
- **Resposta imediata do sistema:** CompareView substitui MapViewer
- **Processamento funcional:** `mapStore.compareMode = true`. No App.vue, `<CompareView v-if="mapStore.compareMode" />` e renderizado em vez de `<MapViewer v-else />`
- **Efeito observavel na UI:** Interface de comparacao aparece com barra de controles no topo (selects Antes/Depois, botoes de modo e fechar) e area de swipe abaixo
- **Evidencia esperada:** Screenshot do CompareView
- **Criterio de aceitacao:** CompareView visivel com controles
- **Seletor Playwright:** `page.locator('.compare-view')`

Passo 2 - Selecionar ortomapa "Antes"

- **Acao do ator:** Seleciona ortomapa mais antigo no dropdown "Antes"
- **Contexto visual anterior:** Dropdown "Antes" com placeholder
- **Elemento de UI utilizado:** Select PrimeVue com v-model `leftId`, opcoes `ortoOptions`, classe `.compare-select`, largura 180px
- **Resposta imediata do sistema:** Ortomapa selecionado, tiles carregam no lado esquerdo
- **Processamento funcional:** `leftId` setado. `l-tile-layer` com URL `/api/ortomapas/${leftId}/tile/{z}/{x}/{y}.png` renderiza no mapa esquerdo
- **Efeito observavel na UI:** Tiles do ortomapa aparecem no lado esquerdo do swipe
- **Evidencia esperada:** Screenshot com lado esquerdo preenchido
- **Criterio de aceitacao:** Tiles visiveis no lado esquerdo
- **Seletor Playwright:** `page.locator('.compare-view .selector-group').first().locator('.p-select')`

Passo 3 - Selecionar ortomapa "Depois"

- **Acao do ator:** Seleciona ortomapa mais recente no dropdown "Depois"
- **Contexto visual anterior:** Dropdown "Depois" com placeholder

- **Elemento de UI utilizado:** Select PrimeVue com v-model `rightId`
- **Resposta imediata do sistema:** Tiles do segundo ortomapa carregam no lado direito
- **Processamento funcional:** `rightId` setado. Segundo mapa renderiza tiles
- **Efeito observavel na UI:** Lado direito mostra segundo ortomapa. Slider no meio (50%)
- **Evidencia esperada:** Screenshot com ambos lados
- **Criterio de aceitacao:** Ambos ortomapas visiveis, slider no centro
- **Seletor Playwright:** `page.locator('.compare-view .selector-group').last().locator('.p-select')`

Passo 4 - Arrastar o slider para comparar

- **Acao do ator:** Arrasta o slider (barra vertical com icone de setas) para a esquerda
- **Contexto visual anterior:** Slider na posicao 50%, ambos os lados visiveis igualmente
- **Elemento de UI utilizado:** Div `.swipe-slider` com `.slider-handle` (circulo com icone `pi pi-arrows-h`). CSS `clipPath: inset()` controla a visibilidade de cada lado
- **Resposta imediata do sistema:** O lado direito (Depois) se expande, revelando mais do ortomapa mais recente
- **Processamento funcional:** `startDrag` ativa `isDragging`. `onDrag` calcula posicao percentual: `pct = (x / rect.width) * 100`, limitado entre 5% e 95%. `sliderPosition` e atualizado. CSS `clipPath` e recalculado: esquerda `inset(0 ${100-pct}% 0 0)`, direita `inset(0 0 0 ${pct}%)`
- **Efeito observavel na UI:** A barra vertical se move, revelando mais de um ortomapa e menos do outro. Efeito de "cortina" visual
- **Evidencia esperada:** Screenshot com slider deslocado para a esquerda
- **Criterio de aceitacao:** O slider deve responder ao arraste e revelar o ortomapa correspondente
- **Seletor Playwright:** `page.locator('.swipe-slider')`

Passo 5 - Alternar para modo Lado a Lado

- **Acao do ator:** Clica no botao "Deslizar" para alternar para "Lado a Lado"
- **Contexto visual anterior:** Modo swipe ativo
- **Elemento de UI utilizado:** Button PrimeVue com label condicional (`swipeMode ? 'Deslizar' : 'Lado a Lado'`), icon condicional, severity `secondary`
- **Resposta imediata do sistema:** A interface muda de swipe para dois paineis lado a lado
- **Processamento funcional:** `swipeMode = !swipeMode` (toggle). Quando false, o template renderiza `.side-by-side` com dois `.side-panel` em vez de `.swipe-container`
- **Efeito observavel na UI:** Dois mapas separados por borda verde, cada um com label "Antes"/"Depois"
- **Evidencia esperada:** Screenshot modo lado a lado
- **Criterio de aceitacao:** Dois paineis de mapa visiveis, cada um com seu ortomapa
- **Seletor Playwright:** `page.locator('.compare-view .compare-mode-toggle button').first()`

Passo 6 - Fechar comparacao

- **Acao do ator:** Clica no botao "Fechar"
- **Contexto visual anterior:** Modo de comparacao ativo
- **Elemento de UI utilizado:** Button "Fechar" com icon `pi pi-times`, severity `danger`
- **Resposta imediata do sistema:** CompareView desaparece, MapViewer volta
- **Processamento funcional:** `mapStore.setCompareMode(false)`. App.vue renderiza MapViewer em vez de CompareView
- **Efeito observavel na UI:** Mapa normal restaurado
- **Evidencia esperada:** Screenshot mapa normal
- **Criterio de aceitacao:** `mapStore.compareMode === false`. MapViewer visivel
- **Seletor Playwright:** `page.locator('.compare-view .compare-mode-toggle button').filter({ hasText: 'Fechar' })`

Fluxos Alternativos

FA-1: Iniciar diretamente em modo Lado a Lado

- **Acao:** Clicar no botao de toggle antes de selecionar ortomapas
- **Efeito:** Dois paineis vazios com mapa base OSM, aguardando selecao

Fluxos de Excecao

FE-1: Tiles nao carregam

- **Condicao:** Endpoint de tiles do ortomapa retorna 404
- **Efeito:** Um ou ambos os lados mostram apenas mapa base sem ortomapa

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.compare-view .selector-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.compare-view .selector-group').last().locator('.p-select').click()`
  - `page.locator('.p-select-option').last().click()`
  - `page.locator('.swipe-slider').dragTo(page.locator('.swipe-container'), { targetPosition: { x: 200, y: 300 } })`
  - `page.locator('.compare-mode-toggle button').first().click()`
  - `page.locator('.compare-mode-toggle button').filter({ hasText: 'Fechar' }).click()`
- **Asserts:**
  - `await expect(page.locator('.compare-view')).toBeVisible()` (apos ativacao)
  - `await expect(page.locator('.side-by-side')).toBeVisible()` (apos toggle)
  - `await expect(page.locator('.compare-view')).toBeHidden()` (apos fechar)
- **Screenshots:**
  - `screenshot_uc017_01_compare_view.png`
  - `screenshot_uc017_02_swipe_ortomapas.png`
  - `screenshot_uc017_03_slider_arrastado.png`
  - `screenshot_uc017_04_lado_a_lado.png`

UC-018: Exportar Ortomapa em Formato Diferente

Identificacao

- **ID:** UC-018
- **Modulo:** Exportacao
- **Prioridade:** Alta
- **Ator:** Usuario

Objetivo

Exportar um ortomapa ou camada de analise para um formato diferente (GeoTIFF, PNG, JPEG, KML, GeoJSON, Shapefile) com sistema de coordenadas configuravel, para uso em outros softwares GIS ou relatorios.

Pre-condicoes

- Projeto ativo com ortomapas e/ou camadas de analise

- Sidebar direita visivel (ou ExportDialog acessivel)

Pos-condicoes

- Arquivo exportado baixado pelo navegador
- Formato e CRS corretos conforme selecionado

Elementos de UI Envolvidos

Via ToolsPanel (aba Exp):

- Tab "Exp" no ToolsPanel
- Select "Camada / Ortomapa" (allLayerOptions: ortomapas + analises)
- Select "Formato" (GeoTIFF, PNG, JPEG, KML, GeoJSON, Shapefile)
- Select "CRS" (EPSG:4326, EPSG:31983, EPSG:31984, EPSG:32723, EPSG:32724)
- Button "Exportar"

Via ExportDialog (modal alternativo):

- Modal `.modal` com selects nativos, input de resolucao, slider de qualidade
- Button `.btn-export`

Fluxo Principal

Passo 1 - Navegar para aba Exportar no ToolsPanel

- **Acao do ator:** Clica na aba "Exp"
- **Contexto visual anterior:** ToolsPanel em outra aba
- **Elemento de UI utilizado:** Tab com valor `exportar`, conteudo `<i class="pi pi-download"></i> Exp`
- **Resposta imediata do sistema:** Formulario de exportacao
- **Processamento funcional:** `activeTab = 'exportar'`
- **Efeito observavel na UI:** Tres selects (camada, formato, CRS) e botao "Exportar"
- **Evidencia esperada:** Screenshot aba exportar
- **Criterio de aceitacao:** Formulario visivel com todos os campos
- **Seletor Playwright:** `page.locator('.tools-panel [data-p-value="exportar"]')`

Passo 2 - Selecionar camada para exportar

- **Acao do ator:** Abre dropdown "Camada / Ortomapa" e seleciona um ortomapa
- **Contexto visual anterior:** Dropdown com placeholder
- **Elemento de UI utilizado:** Select `exportar.layerId` com opcoes `allLayerOptions` (combinacao de ortomapas e camadas de analise)
- **Resposta imediata do sistema:** Camada selecionada
- **Processamento funcional:** `exportar.layerId` setado
- **Efeito observavel na UI:** Nome da camada no dropdown
- **Evidencia esperada:** Screenshot com camada selecionada
- **Criterio de aceitacao:** ID valido selecionado
- **Seletor Playwright:** `page.locator('.tools-panel [value="exportar"] .form-group').first().locator('.p-select')`

Passo 3 - Selecionar formato de saida

- **Acao do ator:** Seleciona "PNG" no dropdown de formato
- **Contexto visual anterior:** Dropdown mostrando "GeoTIFF" (padrao)

- **Elemento de UI utilizado:** Select `exportar.format` com opcoes `exportFormats`
- **Resposta imediata do sistema:** Formato selecionado
- **Processamento funcional:** `exportar.format = 'png'`
- **Efeito observavel na UI:** Dropdown mostra "PNG"
- **Evidencia esperada:** Screenshot com formato selecionado
- **Criterio de aceitacao:** Formato deve ser 'png'
- **Seletor Playwright:** `page.locator('.tools-panel [value="exportar"] .form-group').nth(1).locator('.p-select')`

Passo 4 - Selecionar CRS de saida

- **Acao do ator:** Seleciona "EPSG:31983 (SIRGAS 2000 / UTM 23S)"
- **Contexto visual anterior:** Dropdown mostrando "EPSG:4326 (WGS 84)" (padrao)
- **Elemento de UI utilizado:** Select `exportar.crs` com opcoes `crsOptions`
- **Resposta imediata do sistema:** CRS selecionado
- **Processamento funcional:** `exportar.crs = 'EPSG:31983'`
- **Efeito observavel na UI:** Dropdown mostra "EPSG:31983 (SIRGAS 2000 / UTM 23S)"
- **Evidencia esperada:** Screenshot com CRS selecionado
- **Criterio de aceitacao:** CRS deve ser 'EPSG:31983'
- **Seletor Playwright:** `page.locator('.tools-panel [value="exportar"] .form-group').nth(2).locator('.p-select')`

Passo 5 - Clicar em "Exportar"

- **Acao do ator:** Clica no botao "Exportar"
- **Contexto visual anterior:** Formulario completo
- **Elemento de UI utilizado:** Button "Exportar" com icon `pi pi-download`
- **Resposta imediata do sistema:** Botao em loading
- **Processamento funcional:** (1) Valida `exportar.layerId`. (2) Chama `exportLayer({layer_id, format: 'png', crs: 'EPSG:31983'})` que faz POST `/api/tools/exportar` com `responseType: 'blob'`. (3) Backend: le o raster, reprojecta para CRS destino, converte para formato solicitado. (4) Retorna blob binario. (5) Frontend cria URL blob, cria link `<a>` invisivel, seta `download` com nome `export_timestamp.png`, clica e revoga URL
- **Efeito observavel na UI:** Botao com spinner, depois download automatico do navegador inicia
- **Evidencia esperada:** Screenshot do download iniciando
- **Criterio de aceitacao:** Arquivo deve ser baixado pelo navegador. Nome deve conter "export_" e extensao correta (.png)
- **Seletor Playwright:** `page.locator('.tools-panel [value="exportar"] button').filter({ hasText: 'Exportar' })`

Passo 6 - Verificar download

- **Acao do ator:** Verifica o arquivo baixado
- **Contexto visual anterior:** Download concluido
- **Elemento de UI utilizado:** Navegador (download manager)
- **Resposta imediata do sistema:** Arquivo disponivel no diretorio de downloads
- **Processamento funcional:** Arquivo binario no formato correto
- **Efeito observavel na UI:** Botao volta ao estado normal
- **Evidencia esperada:** Arquivo baixado com formato e extensao corretos
- **Criterio de aceitacao:** Arquivo PNG valido, com CRS EPSG:31983 se aplicavel (metadados)
- **Seletor Playwright:** `page.waitForEvent('download')`

Fluxos Alternativos

FA-1: Exportar via ExportDialog modal

- **Acao:** Usar o ExportDialog que oferece opcoes adicionais de resolucao e qualidade
- **Diferenca:** Modal com campos extras: resolucao (metros), qualidade (slider 10-100% para PNG/JPEG)
- **Seletor Playwright:** `page.locator('.modal .btn-export')`

FA-2: Exportar resultado de analise

- **Passo divergente:** Passo 2
- **Acao:** Selecionar uma camada de analise em vez de ortomapa
- **Efeito:** A exportacao e feita a partir da camada de resultado de analise

Fluxos de Excecao

FE-1: Camada nao selecionada

- **Condicao:** `exportar.layerId` e null
- **Processamento funcional:** Retorno silencioso

FE-2: Formato incompativel com tipo de dados

- **Condicao:** Tentar exportar dados vetoriais como GeoTIFF
- **Processamento funcional:** Backend retorna erro
- **Efeito observavel na UI:** Erro logado, download nao inicia

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.tools-panel [data-p-value="exportar"]').click()`
  - `page.locator('.tools-panel [value="exportar"] .form-group').first().locator('.p-select').click()`
  - `page.locator('.p-select-option').first().click()`
  - `page.locator('.tools-panel [value="exportar"] .form-group').nth(1).locator('.p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'PNG' }).click()`
  - `page.locator('.tools-panel [value="exportar"] .form-group').nth(2).locator('.p-select').click()`
  - `page.locator('.p-select-option').filter({ hasText: 'EPSG:31983' }).click()`
  - `const downloadPromise = page.waitForEvent('download')`
  - `page.locator('.tools-panel [value="exportar"] button').filter({ hasText: 'Exportar' }).click()`
  - `const download = await downloadPromise`
- **Asserts:**
  - `expect(download.suggestedFilename()).toContain('.png')`
- **Screenshots:**
  - `screenshot_uc018_01_aba_exportar.png`
  - `screenshot_uc018_02_formulario_preenchido.png`
  - `screenshot_uc018_03_download.png`

UC-019: Registrar Voo de Drone

Identificacao

- **ID:** UC-019

- **Modulo:** Projetos
- **Prioridade:** Media
- **Ator:** Usuario

Objetivo

Registrar um voo de drone no sistema vinculado ao projeto ativo, documentando data, equipamento, parametros de voo (altitude, numero de fotos, area coberta, GSD) e observacoes, para manter rastreabilidade dos dados coletados.

Pre-condicoes

- Um projeto esta selecionado (ativo)
- Sidebar esquerda visivel com VoosList

Pos-condicoes

- Registro de voo criado no banco de dados vinculado ao projeto
- Voo aparece na lista da secao "Voos"
- Metadados de voo disponiveis para consulta

Elementos de UI Envolvidos

- Secao "Voos" (VoosList) na sidebar esquerda
- Botao "Novo" (severity success, icon pi-plus)
- Dialog modal "Novo Voo" (420px)
- InputText "Data do Voo" (type date)
- InputText "Drone" (placeholder "Ex: DJI Mini 3")
- InputText "Altitude de voo (m)" (type number)
- InputText "Numero de fotos" (type number)
- InputText "Area coberta (ha)" (type number, step 0.1)
- InputText "GSD (cm/px)" (type number, step 0.01)
- Textarea "Observacoes" (2 linhas)
- Botoes "Cancelar" e "Criar"
- Lista `.voo-item` com icone, data, drone, altitude, fotos

Fluxo Principal

Passo 1 - Abrir dialog de novo voo

- **Acao do ator:** Clica no botao "Novo" na secao "Voos"
- **Contexto visual anterior:** Secao Voos visivel com lista de voos ou mensagem vazia. Botao "Novo" habilitado (projeto ativo selecionado)
- **Elemento de UI utilizado:** Button PrimeVue com label "Novo", icon `pi pi-plus`, severity `success`, tamanho `small`, desabilitado se `!projectStore.activeProject`
- **Resposta imediata do sistema:** Dialog modal "Novo Voo" abre
- **Processamento funcional:** `showNewVoo = true`
- **Efeito observavel na UI:** Modal com 7 campos de formulario
- **Evidencia esperada:** Screenshot do dialog aberto
- **Criterio de aceitacao:** Dialog visivel com todos os campos vazios
- **Seletor Playwright:** `page.locator('.voos-list .section-header button').filter({ hasText: 'Novo' })`

Passo 2 - Preencher data do voo

- **Acao do ator:** Seleciona a data "2026-03-15" no campo de data
- **Contexto visual anterior:** Campo de data vazio
- **Elemento de UI utilizado:** InputText PrimeVue com type `date`, v-model `newVoo.data_voo`
- **Resposta imediata do sistema:** Data selecionada exibida
- **Processamento funcional:** `newVoo.data_voo = '2026-03-15'`
- **Efeito observavel na UI:** Campo mostra "2026-03-15"
- **Evidencia esperada:** Screenshot com data
- **Criterio de aceitacao:** Data deve ser '2026-03-15'
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').first().locator('input')`

Passo 3 - Preencher informacoes do drone

- **Acao do ator:** Digita "DJI Mini 3" no campo "Drone"
- **Contexto visual anterior:** Campo drone vazio
- **Elemento de UI utilizado:** InputText com placeholder "Ex: DJI Mini 3"
- **Resposta imediata do sistema:** Texto digitado
- **Processamento funcional:** `newVoo.drone = 'DJI Mini 3'`
- **Efeito observavel na UI:** Campo mostra "DJI Mini 3"
- **Evidencia esperada:** Screenshot com drone
- **Criterio de aceitacao:** Campo deve conter "DJI Mini 3"
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').nth(1).locator('input')`

Passo 4 - Preencher parametros de voo

- **Acao do ator:** Preenche altitude (120), numero de fotos (256), area coberta (15.5), GSD (2.74)
- **Contexto visual anterior:** Campos numericos em branco
- **Elemento de UI utilizado:** Quatro InputText com type number: altitude (placeholder "120"), fotos, area (step 0.1), GSD (step 0.01)
- **Resposta imediata do sistema:** Valores preenchidos
- **Processamento funcional:** `newVoo.altitude_voo = 120`, `newVoo.num_fotos = 256`, `newVoo.area_coberta_ha = 15.5`, `newVoo.gsd_cm = 2.74`
- **Efeito observavel na UI:** Todos os campos numericos preenchidos
- **Evidencia esperada:** Screenshot formulario completo
- **Criterio de aceitacao:** Todos os valores numericos corretos
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').nth(2).locator('input')` (altitude)

Passo 5 - Adicionar observacoes

- **Acao do ator:** Digita "Voo realizado com condicoes climaticas favoraveis, vento < 10km/h"
- **Contexto visual anterior:** Textarea vazio
- **Elemento de UI utilizado:** Textarea PrimeVue com 2 linhas
- **Resposta imediata do sistema:** Texto aparece
- **Processamento funcional:** `newVoo.observacoes` atualizado
- **Efeito observavel na UI:** Textarea preenchido
- **Evidencia esperada:** Screenshot com observacoes
- **Criterio de aceitacao:** Texto presente
- **Seletor Playwright:** `page.locator('.p-dialog .form-group').last().locator('textarea')`

Passo 6 - Criar o voo

- **Acao do ator:** Clica no botao "Criar"

- **Contexto visual anterior:** Formulario completo
- **Elemento de UI utilizado:** Button "Criar" com icon `pi pi-check` no footer do Dialog
- **Resposta imediata do sistema:** Dialog fecha
- **Processamento funcional:** (1) `addVoo()` chama `createVoo({projeto_id, data_voo, drone, altitude_voo, num_fotos, area_coberta_ha, gsd_cm, observacoes})` via POST `/api/voos`. (2) Backend cria registro. (3) Reseta `newVoo`. (4) `showNewVoo = false`. (5) `fetchVoos()` recarrega lista
- **Efeito observavel na UI:** (1) Dialog fecha. (2) Novo `.voo-item` aparece na lista com data "2026-03-15", drone "DJI Mini 3", altitude "120m", fotos "256"
- **Evidencia esperada:** Screenshot da lista com novo voo
- **Criterio de aceitacao:** Voo deve aparecer na lista com todos os dados corretos. Badge de contagem no titulo "Voos" deve incrementar
- **Seletor Playwright:** `page.locator('.p-dialog-footer button').filter({ hasText: 'Criar' })`

Fluxos Alternativos

FA-1: Cancelar criacao

- **Acao:** Clica "Cancelar"
- **Efeito:** Dialog fecha, nenhum voo criado

Fluxos de Excecao

FE-1: Nenhum projeto ativo

- **Condicao:** `projectStore.activeProject` e null
- **Efeito:** Botao "Novo" esta desabilitado (`:disabled="!projectStore.activeProject"`)

FE-2: Erro ao criar voo

- **Condicao:** Backend retorna erro
- **Processamento funcional:** `console.error('Erro ao criar voo:', e)`
- **Efeito observavel na UI:** Dialog pode permanecer aberto, voo nao aparece na lista

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.voos-list .section-header button').filter({ hasText: 'Novo' }).click()`
  - `page.locator('.p-dialog .form-group').first().locator('input').fill('2026-03-15')`
  - `page.locator('.p-dialog .form-group').nth(1).locator('input').fill('DJI Mini 3')`
  - `page.locator('.p-dialog .form-group').nth(2).locator('input').fill('120')`
  - `page.locator('.p-dialog .form-group').nth(3).locator('input').fill('256')`
  - `page.locator('.p-dialog .form-group').nth(4).locator('input').fill('15.5')`
  - `page.locator('.p-dialog .form-group').nth(5).locator('input').fill('2.74')`
  - `page.locator('.p-dialog .form-group').last().locator('textarea').fill('Condicoes favoraveis')`
  - `page.locator('.p-dialog-footer button').filter({ hasText: 'Criar' }).click()`
- **Asserts:**
  - `await expect(page.locator('.p-dialog')).toBeVisible()` (apos passo 1)
  - `await expect(page.locator('.p-dialog')).toBeHidden()` (apos passo 6)
  - `await expect(page.locator('.voo-item .voo-date')).toContainText('2026-03-15')`
  - `await expect(page.locator('.voo-item .meta-item').first()).toContainText('DJI Mini 3')`
- **Screenshots:**
  - `screenshot_uc019_01_dialog_aberto.png`
  - `screenshot_uc019_02_formulario_preenchido.png`

- `screenshot_uc019_03_voo_na_lista.png`

UC-020: Disparar Analise via Agente de IA

Identificacao

- **ID:** UC-020
- **Modulo:** Analise
- **Prioridade:** Alta
- **Ator:** Usuario / Agente IA

Objetivo

Criar uma analise (ex: classificacao de solo, indice de vegetacao) e enfileira-la para processamento automatico por um agente de IA, acompanhando o status em tempo real na barra de status (AgentStatus) ate a conclusao ou erro.

Pre-condicoes

- Projeto ativo com ortomapas
- AnalysisForm acessivel (via inject `showAnalysisForm`)
- Backend com sistema de filas/agentes funcional

Pos-condicoes

- Analise criada no banco com status "em_fila" ou "processando"
- Agente de IA processa a analise automaticamente
- Status atualizado em tempo real na barra AgentStatus
- Resultado adicionado ao mapa quando concluido

Elementos de UI Envolvidos

- Modal AnalysisForm (`.modal`) com formulario completo
- Select de ortomapa
- Input de nome
- Select de tipo de analise
- Parametros dinamicos por tipo
- Checkbox "Executar via Agente de IA" (`.queue-task-group`)
- Button "Criar e Executar" (`.btn-submit`)
- AgentStatus na barra de status inferior
- Lista de tarefas ativas/historico no AgentStatus

Fluxo Principal

Passo 1 - Abrir o formulario de analise

- **Acao do ator:** Clica no botao "+" na barra de status (AgentStatus) ou no botao "Analisar" de um OrtomapCard
- **Contexto visual anterior:** Barra de status na parte inferior mostrando "Nenhuma tarefa em execucao" e botoes de acao
- **Elemento de UI utilizado:** Button `.status-btn` com icon `pi pi-plus` no AgentStatus, ou `.action-btn` com `pi pi-chart-bar` no OrtomapCard
- **Resposta imediata do sistema:** Modal AnalysisForm aparece centralizado
- **Processamento funcional:** `openAnalysisForm()` seta `showAnalysisForm.value = true` (inject/provide do App.vue)

- **Efeito observavel na UI:** Modal com formulario completo de analise
- **Evidencia esperada:** Screenshot do AnalysisForm modal
- **Criterio de aceitacao:** Modal visivel com titulo "Nova Analise"
- **Seletor Playwright:** `page.locator('.agent-status .status-btn').first()`

Passo 2 - Selecionar ortomapa

- **Acao do ator:** Seleciona um ortomapa no dropdown
- **Contexto visual anterior:** Dropdown "Selecione um ortomapa..." com opcoes dos ortomapas do projeto
- **Elemento de UI utilizado:** Select nativo `<select v-model="form.ortomapa_id">` com options dos ortomapas
- **Resposta imediata do sistema:** Ortomapa selecionado
- **Processamento funcional:** `form.ortomapa_id` setado
- **Efeito observavel na UI:** Nome do ortomapa no dropdown
- **Evidencia esperada:** Screenshot com ortomapa selecionado
- **Criterio de aceitacao:** ID valido selecionado
- **Seletor Playwright:** `page.locator('.modal-body select').first()`

Passo 3 - Preencher nome da analise

- **Acao do ator:** Digita "Classificacao Solo KMeans - Serra Cipo Mar 2026"
- **Contexto visual anterior:** Campo de nome vazio
- **Elemento de UI utilizado:** Input `v-model="form.nome"` com placeholder "Ex: NDVI Fazenda Norte - Mar 2026"
- **Resposta imediata do sistema:** Nome digitado
- **Processamento funcional:** `form.nome` atualizado
- **Efeito observavel na UI:** Campo preenchido
- **Evidencia esperada:** Screenshot com nome
- **Criterio de aceitacao:** Nome deve conter o texto digitado
- **Seletor Playwright:** `page.locator('.modal-body input[placeholder*="NDVI"]')`

Passo 4 - Selecionar tipo e parametros

- **Acao do ator:** Seleciona "Classificacao de Solo" no tipo, mantém KMeans como algoritmo e 5 classes
- **Contexto visual anterior:** Tipo padrao "Indice de Vegetacao"
- **Elemento de UI utilizado:** Select `form.tipo_analise` (nativo), seleciona `'classificacao_solo'`. Campos condicionais de classificacao aparecem: algoritmo (kmeans/random_forest/svm), numero de classes
- **Resposta imediata do sistema:** Campos de parametros de classificacao aparecem (template condicional `v-if="form.tipo_analise === 'classificacao_solo'"`)
- **Processamento funcional:** `form.tipo_analise = 'classificacao_solo'`. `params.algorithm = 'kmeans'`, `params.n_classes = 5`
- **Efeito observavel na UI:** Campos de algoritmo e numero de classes visiveis
- **Evidencia esperada:** Screenshot com tipo e parametros
- **Criterio de aceitacao:** Campos de classificacao visiveis com valores corretos
- **Seletor Playwright:** `page.locator('.modal-body select').nth(1)`

Passo 5 - Ativar execucao via Agente de IA

- **Acao do ator:** Marca o checkbox "Executar via Agente de IA"
- **Contexto visual anterior:** Checkbox desmarcado na secao `.queue-task-group` com borda azul
- **Elemento de UI utilizado:** Checkbox input dentro de `.checkbox-label` com texto "Executar via Agente de IA". Dica: "Quando ativado, a analise sera enfileirada para processamento automatico por agente de IA"
- **Resposta imediata do sistema:** Checkbox marcado
- **Processamento funcional:** `form.queue_task = true`

- **Efeito observavel na UI:** Checkbox com checkmark
- **Evidencia esperada:** Screenshot com checkbox marcado
- **Criterio de aceitacao:** `form.queue_task` deve ser true
- **Seletor Playwright:** `page.locator('.queue-task-group input[type="checkbox"]')`

Passo 6 - Clicar em "Criar e Executar"

- **Acao do ator:** Clica no botao "Criar e Executar"
- **Contexto visual anterior:** Formulario completo, checkbox de agente marcado
- **Elemento de UI utilizado:** Button `.btn-submit` com texto "Criar e Executar"
- **Resposta imediata do sistema:** Botao mostra "Criando..." com spinner
- **Processamento funcional:** (1) Valida ortomapa e nome. (2) Monta dados com `queue_task: true`. (3) POST `/api/analises` com `{ortomapa_id, projeto_id, tipo_analise: 'classificacao_solo', nome: '...', parametros: JSON.stringify({algorithm: 'kmeans', n_classes: 5}), queue_task: true}`. (4) Backend cria registro com status "em_fila". (5) Agente de IA pega a tarefa da fila e inicia processamento. (6) `projectStore.fetchAnalises()` recarrega. (7) Modal fecha. (8) Formulario resetado
- **Efeito observavel na UI:** (1) Botao com spinner "Criando...". (2) Modal fecha. (3) Na barra de status, AgentStatus mostra nova tarefa "em_fila" ou "processando" com icone de relogio/spinner
- **Evidencia esperada:** Screenshot da barra de status com tarefa em processamento
- **Criterio de aceitacao:** A tarefa deve aparecer no AgentStatus. O modal deve fechar. Status deve ser "em_fila" ou "processando"
- **Seletor Playwright:** `page.locator('.modal-footer .btn-submit')`

Passo 7 - Acompanhar status do agente

- **Acao do ator:** Observa a barra de status (AgentStatus) e opcionalmente clica em "Ver X tarefa(s)"
- **Contexto visual anterior:** Barra de status com tarefa ativa
- **Elemento de UI utilizado:** AgentStatus com `.task-item` mostrando nome do agente, tipo da tarefa, tempo decorrido. Botao `.toggle-detail` para expandir detalhes. Painel `.tasks-detail` com lista detalhada
- **Resposta imediata do sistema:** Status atualiza automaticamente a cada 5 segundos (polling)
- **Processamento funcional:** `onMounted` inicia `setInterval` de 5 segundos chamando `projectStore.pollAgentTasks()` (GET `/api/analises?status=em_fila,processando`). Quando a tarefa conclui, seu status muda para "concluida" e o icone muda para check verde
- **Efeito observavel na UI:** (1) Inicialmente: icone relogio azul (em_fila) ou spinner amarelo (processando). (2) Tempo decorrido incrementa. (3) Apos conclusao: icone check verde, badge "Concluida"
- **Evidencia esperada:** Screenshots sequenciais do status mudando
- **Criterio de aceitacao:** Status deve transicionar de "em_fila" para "processando" para "concluida". Tempo decorrido deve ser exibido corretamente
- **Seletor Playwright:** `page.locator('.agent-status .task-item')`

Passo 8 - Verificar resultado no mapa

- **Acao do ator:** Verifica se a camada de resultado apareceu no mapa
- **Contexto visual anterior:** Mapa com camadas anteriores
- **Elemento de UI utilizado:** MapViewer e LayerPanel
- **Resposta imediata do sistema:** Camada de classificacao adicionada ao mapa
- **Processamento funcional:** Quando a analise conclui, o resultado pode ser carregado automaticamente como camada
- **Efeito observavel na UI:** Mapa classificado com cores distintas por classe de uso do solo
- **Evidencia esperada:** Screenshot do mapa com resultado da analise
- **Criterio de aceitacao:** Camada de resultado visivel no mapa
- **Seletor Playwright:** `page.locator('.map-container')`

Fluxos Alternativos

FA-1: Executar sem agente (diretamente)

- **Passo divergente:** Passo 5
- **Acao:** Manter checkbox desmarcado
- **Processamento:** `queue_task: false`. A analise e executada imediatamente pelo backend, sem enfileirar
- **Efeito:** Resultado retornado diretamente na resposta da API. Modal fecha com resultado imediato

FA-2: Expandir detalhes das tarefas

- **Passo divergente:** Passo 7
- **Acao:** Clica em "Ver X tarefa(s)"
- **Efeito:** Painel `.tasks-detail` aparece como dropdown com lista detalhada de todas as tarefas, incluindo historico com status (badge colorido), nome, agente, tempo e erros
- **Seletor Playwright:** `page.locator('.agent-status .toggle-detail')`

Fluxos de Excecao

FE-1: Campos obrigatorios nao preenchidos

- **Condicao:** Ortomapa ou nome nao preenchidos
- **Processamento funcional:** `if (!form.ortomapa_id || !form.nome)` seta `error.value = 'Preencha o ortomapa e o nome da analise.'`
- **Efeito observavel na UI:** Mensagem de erro vermelha aparece: "Preencha o ortomapa e o nome da analise." na div `.error-message`
- **Criterio de aceitacao:** Mensagem de erro visivel, modal nao fecha
- **Seletor Playwright:** `page.locator('.modal-body .error-message')`

FE-2: Erro do backend ao criar analise

- **Condicao:** Backend retorna erro 500 ou 422
- **Processamento funcional:** Catch captura erro, `error.value = e.response?.data?.detail || e.message`
- **Efeito observavel na UI:** Mensagem de erro vermelha com icone `pi-exclamation-triangle` e texto do erro

FE-3: Agente de IA falha no processamento

- **Condicao:** O agente encontra erro durante processamento
- **Processamento funcional:** Status da analise muda para "erro". `task.erro` contem descricao do problema
- **Efeito observavel na UI:** No AgentStatus, icone triangulo vermelho, badge "Erro". No painel detalhado, mensagem de erro em fundo rosa

Mapeamento Playwright

- **Acoes:**
  - `page.locator('.agent-status .status-btn').first().click()`
  - `page.locator('.modal-body select').first().selectOption({ index: 1 })`
  - `page.locator('.modal-body input[placeholder*="NDVI"]').fill('Classificacao Solo KMeans - Serra Cipo Mar 2026')`
  - `page.locator('.modal-body select').nth(1).selectOption('classificacao_solo')`
  - `page.locator('.queue-task-group input[type="checkbox"]').check()`
  - `page.locator('.modal-footer .btn-submit').click()`
  - `page.waitForSelector('.agent-status .task-item')`
  - `page.locator('.agent-status .toggle-detail').click()`

- **Asserts:**
  - `await expect(page.locator('.modal')).toBeVisible()` (apos passo 1)
  - `await expect(page.locator('.modal')).toBeHidden()` (apos passo 6)
  - `await expect(page.locator('.agent-status .task-item')).toBeVisible()` (apos passo 6)
  - `await expect(page.locator('.agent-status .tasks-detail')).toBeVisible()` (apos expandir)
- **Screenshots:**
  - `screenshot_uc020_01_analysis_form.png`
  - `screenshot_uc020_02_formulario_preenchido.png`
  - `screenshot_uc020_03_checkbox_agente.png`
  - `screenshot_uc020_04_loading.png`
  - `screenshot_uc020_05_tarefa_em_fila.png`
  - `screenshot_uc020_06_tarefa_processando.png`
  - `screenshot_uc020_07_tarefa_concluida.png`
  - `screenshot_uc020_08_resultado_mapa.png`

Adendo normativo da versao 2.0

Regra de classificacao

Um caso de uso somente pode ser marcado como **Aderente** quando possui: (a) requisito descrito; (b) endpoint ou servico implementado; (c) acionador visivel na interface; (d) resultado observavel; e (e) teste Playwright ou teste de aceite equivalente. “Endpoint existente” isoladamente e classificado como **Parcial**.

Matriz oficial de aderencia da interface

ID	Status v2.0	Fundamentacao
UC-001	Aderente	Dialog Novo, POST/GET e lista persistida
UC-002	Aderente	Busca global da toolbar executa endpoint autenticado, filtra a lista, permite abrir projeto, informa zero resultados e restaura a lista ao limpar; Playwright com dois projetos Aukerman reais em 06/11/2026.
UC-003	Aderente	Selecao de projeto, cards e camadas
UC-004	Aderente	Formulario Importar GeoTIFF, metadados, thumbnail e ativacao automatica no mapa validados via Playwright; arquivo invalido rejeitado com HTTP 400.
UC-005	Aderente	Aba Veg e indices RGB
UC-006	Aderente	Aba Terreno, slope/aspect
UC-007	Aderente	Aba Terreno, intervalo e GeoJSON
UC-008	Aderente	Aba Terreno, hillshade
UC-009	Aderente	Aba Mudancas, dois produtos e limiar
UC-010	Aderente	Aba Classificacao e parametros

ID	Status v2.0	Fundamentacao
UC-011	Aderente	Aba Hidrologia e ponto de exutorio
UC-012	Aderente	Aba Volume e cota de referencia
UC-013	Aderente	DrawTools poligono + formulario
UC-014	Aderente	DrawTools ponto + formulario
UC-015	Aderente	Comando visivel no mapa; linha medida, segmentos, desfazer e limpeza validados via Playwright em 06/11/2026.
UC-016	Aderente	Comando visivel no mapa; poligono, area/perimetro, limpar e fechar validados via Playwright em 06/11/2026.
UC-017	Aderente	Comando Comparar no mapa; duas ortofotos selecionadas, modos deslizante/lado a lado e retorno validados via Playwright em 06/11/2026.
UC-018	Aderente	Aba Exp, formato, CRS e download
UC-019	Aderente	Dialog de voo, fotos e configuracao ODM
UC-020	Parcial (escopo de fila limitado)	K-Means RGB, índices RGB, slope/aspect/hillshade, curvas, rede D8 e bacia por exutório passaram pelas evidências de 06/11. Na etapa 9.1, direção D8, acumulação D8 e TWI foram conectados à fila e processados pelo worker sobre o DTM ODM real #21 (EPSG:32617); os artefatos e checksums estão em `docs/validacao/ortomapas_conclusao/etapa-09/uc020_dtm_real.json`. Suíte do worker usa banco/publicador simulados; API e UI com banco real ainda aguardam validação Playwright. A bacia é sensível à escala e requer calibração externa. Permanecem mudança, volume, segmentação e classificadores supervisionados na fila; também falta completar o aceite visual/API dos três produtos novos.

Alteracoes obrigatorias para fechar os parciais

1. Concluido em 06/11/2026: busca global conectada ao endpoint e à lista, incluindo estado sem resultados e limpeza; ver relatorio e evidencias UC-002.
2. Concluido em 06/11/2026: upload GeoTIFF via UI, metadata, thumbnail, selecao de tipo e camada visivel; ver relatorio e evidencias UC-004.
3. Concluido em 06/11/2026: barra de medicao no mapa, estado compartilhado, pontos, linha/poligono, desfazer e limpar; ver relatorio Playwright UC-015/016.
4. Concluido em 06/11/2026: comando Comparar, seleção inicial de produtos do mesmo tipo e retorno ao mapa; ver relatorio Playwright UC-017.
5. Concluido em 06/11/2026: diagrama de atores, casos de uso e componentes.
6. Expandir UC-020 para os outros tipos de análise enfileirável descritos. K-Means RGB, índices RGB, slope/aspect, hillshade, curvas, rede D8 e bacia por exutório passaram pelo fluxo de fila. Continuam pendentes direção/acumulação autônomas, TWI, mudanças, volume, segmentação e classificadores supervisionados; também falta calibrar a bacia contra referência independente.

Criterio de aceite da v2.0

Um item permanece parcial na matriz: o escopo ampliado do UC-020, que deve ser validado conforme seus criterios antes do aceite final. No aceite de 29/10/2026, UC-020 era bloqueante: uma chamada de aceite ao endpoint com projeto `62`, ortomapa `42` e `queue_task=true` retornou HTTP 422 (`Fila de analise nao disponivel neste endpoint`). A contagem de `analises` permaneceu 98 e a de `tarefas_agentes` permaneceu 0, antes e depois. Isso prova que a tarefa nao foi enfileirada e que nenhuma gravacao ocorreu. O teste de chat do Copiloto valida outra capacidade e nao deve ser usado como evidência de UC-020. A suite anterior de 17/17 aprovacoes continua valida somente para os fluxos cobertos, sem promover automaticamente casos parciais.

Adendo de revalidação — 06/11/2026

Na primeira revalidação de 06/11, o POST retornou HTTP 202 e três tarefas K-Means RGB concluíram; o raster da análise 120 foi exibido com 26 tiles HTTP 200 e sem erros JavaScript. Naquele ponto, apenas K-Means estava coberto pela fila. As extensões posteriores do mesmo dia estão registradas abaixo e ampliaram o escopo. Consulte relatório da etapa K-Means. A rejeição do DTM de 1 banda naquela validação foi comportamento correto.

Adendo — validação UC-015/016 em 06/11/2026

As ferramentas de distância e área foram adicionadas à barra do mapa e validadas visualmente com Playwright: três pontos produziram dois segmentos e 3,509 km; desfazer removeu um segmento; quatro vértices geraram polígono, área e perímetro; limpar e fechar restauraram o mapa. Foi removida uma duplicação do painel no workspace de dados. Consulte relatório e capturas UC-015/016.

Adendo — validação UC-017 em 06/11/2026

O comando Comparar foi adicionado à barra do mapa. Playwright abriu o comparador, selecionou duas ortofotos Aukerman do mesmo tipo, carregou tiles HTTP 200 em modos deslizante e lado a lado e fechou retornando ao mapa. Ver relatório e evidências UC-017. O diagrama de atores e casos de uso foi versionado em diagrama_atores_casos_de_uso_v2.mmd.

Adendo — revalidação UC-020, fila mult tipo, em 06/11/2026

A fila foi ampliada para classificação RGB K-Means e índices RGB VARI, TGI, ExG e GLI. Em fluxo iniciado pela interface com raster de demonstração sintético reprojetado, Playwright enfileirou K-Means com 3 grupos e VARI. As análises 128 e 129 concluíram, os resultados foram publicados como GeoTIFF e carregados no mapa; 75 respostas de tile foram HTTP 200, sem erros JavaScript. O painel AgentStatus apresentou as duas tarefas concluídas. O conjunto temporário foi removido e a limpeza foi confirmada no PostgreSQL. Ver relatório e capturas do lote RGB.

Em seguida, a fila foi ampliada com declividade e orientação a partir de DSM/DTM. Playwright usou o DSM ODM real já cadastrado no projeto Aukerman (7.448 × 5.907, banda única, EPSG:32617): as análises 130 e 131 concluíram, 18 tiles foram HTTP 200 e não houve erros JavaScript. A execução visual capturou o estado inicial `em_fila`, o estado `processando` no AgentStatus e a conclusão. Ver relatório e capturas de terreno.

Na sequência, hillshade e curvas de nível foram implementados na fila e executados sobre o mesmo DSM real. As análises 132 e 133 concluíram; o mapa serviu 25 tiles de hillshade HTTP 200 e renderizou 1.573 feições de curvas de nível. Entradas inválidas foram recusadas com HTTP 422 sem gravar tarefa, e não houve erros JavaScript. O relatório contém parâmetros, resultados, critérios e capturas.

A fila hidrológica foi estendida para rede potencial de drenagem D8. Playwright usou o DTM real produto 36, EPSG:32617, com área contribuinte mínima de 1.000 m². A análise 138/tarefa 21 concluiu, publicou GeoJSON com 8 feições e a tela mostrou os 8 caminhos vetoriais. A API reiniciada registrou o agente hidrológico; a tarefa 23/análise 140 concluiu depois do reinício. As recusas de entradas incompatíveis e área inválida retornaram HTTP 422. A precisão da drenagem não foi comparada a levantamento de campo. Consulte relatório da fila hidrológica.

Adendo — bacia por exutório no mapa, UC-020, 06/11/2026

A interface passou a oferecer escolha de exutório diretamente no mapa e enfileiramento da delimitação D8. Ao iniciar a seleção, carrega e enquadra automaticamente o DTM e mostra uma instrução de clique. A análise 143/tarefa 26 usou o DTM ODM real 36, concluiu em 5,39 s, publicou GeoTIFF 406 × 512 em EPSG:32617 com 1.711 células (905,16 m²), exibiu a máscara e o marcador e recebeu 25 respostas HTTP 200 nos tiles da análise. Exutório ausente e raio acima de 1.000 m foram recusados com HTTP 422 sem novas tarefas. A calibração comparativa encontrou forte sensibilidade à grade; a área é demonstrativa, não certificada para decisão técnica. As análises de calibração 141/142 e seus arquivos foram removidos, preservando a auditoria das tarefas. Evidências e limitações estão no relatório detalhado.

Resumo atualizado na etapa 9.1: UC-020 permanece **parcial**. A fila também calcula direção D8, acumulação D8 e TWI e os workers foram executados sobre o DTM real #21; ainda não houve teste visual via API/DB ativo. Permanecem mudança, volume, segmentação e classificadores supervisionados fora da fila. A estimativa da bacia não foi aferida em campo. Consulte o relatório UC-020 e a evidência com o DTM real da etapa 9.1.

Adendo — validação UC-002 em 06/11/2026

A busca da barra superior foi ligada ao endpoint autenticado `/api/projetos/search` e os resultados passaram a controlar a lista de projetos. Playwright validou dois projetos Aukerman retornados por HTTP 200, abertura do projeto escolhido, mensagem explícita para termo sem resultados e restauração dos 51 projetos ao limpar a busca; não houve erros JavaScript. Ver relatório e capturas UC-002.

Adendo — validacao UC-004 em 06/11/2026

O upload ganhou formulario proprio na secao Ortomapas, metadados de sucesso e ativacao automatica no mapa. Playwright criou projeto temporario, recusou arquivo `.txt` com HTTP 400, importou o GeoTIFF demo Guarajuba (256×256, 3 bandas, EPSG:4326) sem voo, recebeu HTTP 201, carregou thumbnail PNG de 512 px e 25 tiles HTTP 200. Projeto, registro e arquivos de teste foram removidos ao final. A rodada corrigiu a obrigatoriedade indevida de `voo_id` no schema PostgreSQL e a ausencia de proxy Vite para `/data`. O raster de exemplo e sintetico e serve para demonstrar a importacao; nao representa foto capturada por drone. Ver relatorio e evidencias UC-004.